

Is an RL Compaction Trigger for RocksDB Worth It?

A Theoretical Feasibility Analysis — Revision 4

static level targets, direct I/O and WAL on, full read-cost model, RocksDB picks every compaction

Feasibility report (draft for EDBT 2027 planning)

September 2026

Abstract

We analyse five requirements for a reinforcement-learning (RL) controller that decides when RocksDB compacts, under the project’s ground rules: static level targets (`level_compaction_dynamic_level_bytes=false`); L0 native except its file trigger; RocksDB’s own picker choosing every compaction, the controller changing only the L0 trigger and per-level target multipliers; a Facebook-derived `mixgraph` workload; and direct I/O plus WAL for all reported results. Every result is tagged **[PROVEN]** (within the stated model), **[CONDITIONAL: assumption]** (under a named assumption), **[EMPIRICAL]** (rests on cited measurements), **[VERIFIED: source]** (a RocksDB detail checked in the source tree, `main` branch, September 2026) or **[VERIFY: item]** (still open). Revision 4 records the project’s decisions: the phase-aware SLO is the objective, every run starts cold, and the learner itself decides which levels are worth learning (Proposition 5.21). It centres the SLO box on a knee the controller can reach, derives the space budget from that box, and states which results depend on the measurement mode (Appendix B, which also keeps Revision 3’s changes). The main consequences are: (i) with static targets, the level above a partly filled last level carries a *first-order* write imbalance; in the worked example a well-chosen static configuration captures almost all of the fix, while `dynamic_level_bytes=true` trades writes for much lower space amplification, so comparisons must be against the static hull; (ii) a per-level controller at level i needs data proportional to that level’s size, so in the project’s 10M runs only the top one or two levels at $T=10$ can learn at all; deeper levels can be set without learning by a propagation law, whose modelled gain over the best static configuration with the same number of levels is small in the worked example (0.8–3.7% for the law as it would be deployed) unless much garbage reaches deep levels; (iii) decision staleness can be removed exactly by deciding where RocksDB computes compaction scores, which it does under the DB mutex; (iv) a Double-DQN on discrete actions is sufficient under the conditions of Section 10, and DDPG would not fix any observed failure; the recommended learner is its one-step form with a model-based prior (Section 13); (v) lower amplification does not imply lower latency, so latency must be measured; (vi) RocksDB also scores L0 by bytes, so the effective L0 trigger is $\min(k, C_1/f_0)$, which removes several points of the planned configuration grid.

Contents

1	Ground rules, baselines and metrics	4
2	The experimental configuration and what it implies	5
3	Preliminaries	6
3.1	Notation	6

3.2	RocksDB mechanics under static targets	6
3.3	Write cost	7
3.4	Read cost: the full point-lookup model	7
3.5	Space	8
4	Requirement 1: compact / defer / defer+expand, with decay	8
4.1	MDP formalisation	8
4.2	Space cost of temporary expansion	9
4.3	Write effect: the generalised conservation identity	9
4.4	Read effect of expansion	11
4.5	Decay schemes and stability	11
5	Requirement 2: per-level control, learned where the data allow	12
5.1	How much data a level- i controller needs	12
5.2	Coupling between controllers	14
5.3	Safety envelope under RocksDB's picker	14
5.4	Decision staleness	16
5.5	Setting deep levels without controllers: a propagation law	16
5.6	Validation status and plan	20
5.7	Which levels to learn: a test the learner can run	21
6	Requirement 3: what each controller observes	22
6.1	Two kinds of coupling	23
6.2	Scenario A: the propagation model is validated	23
6.3	Scenario B: the model fails; an agent for the top levels, deep levels native	24
7	Requirement 4: proactive vs. reactive triggering	26
7.1	Dynamic policies live in the convex hull	27
8	Requirement 5: user-facing SLO with phase detection	28
8.1	Formalisation	28
8.2	Phase detection	29
8.3	Break-even phase length	29
9	Latency, measurement modes and the WAL	30
9.1	Lower amplification does not imply lower latency	30
9.2	Paper mode vs. iteration mode	31
9.3	Write amplification and the WAL	31
10	Learner choice: is a DQN enough?	31
10.1	Exploration at a cold start	32
11	Empirical context from the previous project	33
12	Overall verdict	33
13	Recommended architecture	34
14	Threats to validity	34

A	Assumption register	35
B	Changes	37
B.1	Changes in Revision 4 (from Revision 3)	37
B.2	Changes in Revision 3 (from Revision 2)	38
B.3	Changes in Revision 2 (from Revision 1)	39

1 Ground rules, baselines and metrics

Assumption 1.1 (Ground rules, fixed by the project). **G1** Leveled compaction with file-level (partial) compactions; `compaction_pri=kMinOverlappingRatio` (RocksDB’s default).

G2 `level_compaction_dynamic_level_bytes=false`: static level targets (Section 3.2).

G3 RocksDB otherwise near default. L0 stays native except for `level0_file_num_compaction_trigger` (k): the controller may set k , but L0→L1 compaction itself is unchanged. Because RocksDB also scores L0 by bytes, the trigger that takes effect is $k_{\text{eff}} = \min(k, \lceil C_1/f_0 \rceil)$ (E10).

G4 Workload: `db_bench mixgraph` parameterised from Cao et al. [26], modified so key–value sizes barely vary (64 B keys, mean value ≈ 960 B). It issues gets, puts and seeks, and no deletes.

G5 Paper mode: direct I/O on for user reads and for flush/compaction; WAL on with sync off. Iteration mode (direct I/O off, WAL off) is for debugging the harness only; no conclusion is drawn from it. Section 9.2 lists which results depend on the mode.

G6 The RL controller starts cold (untrained) in every run; nothing learned is carried from one run to the next (decision O2, Section 13).

G7 RocksDB picks every compaction: which level and which files. The controller changes only inputs that RocksDB scores against, namely k and a target multiplier m_i per controlled level (effective target $m_i C_i$; $m_i < 1$ compacts level i early, $m_i > 1$ defers or expands it). A source patch applies m_i to one level only and allows fractional values (Remark 4.13); the picker is not patched.

Baselines. (B1) native RocksDB with the same configuration; (B2) native RocksDB with `dynamic_level_bytes=true`, otherwise identical, reported as a *reference row only*, not a target: it is RocksDB’s default since 8.4 [28, 32], and it fixes every size ratio at T , which bounds space amplification but, in the worked example of Section 5.5, costs more writes than static targets (Table 1); (B3) the lower convex hull of static-target configurations over T , k_{eff} and `max_bytes_for_level_base` (Section 7.1 explains why the hull, not single points). Claims are made against B1 and B3; B2 shows what RocksDB’s default trades for space.

Metrics. (M1) Compaction write amplification

$$\text{WA}_c = \frac{\text{flush bytes} + \text{compaction bytes written}}{\text{user bytes}};$$

WAL bytes are reported separately (Section 9.3). (M2) Point-read amplification $\text{RA} = \text{blocks read from storage per point lookup}$: data-block cache misses plus the reads needed to open tables (Theorem 3.2); in paper mode these are device reads. The controller optimises its data-block part RA_d and reports table opens separately (Section 3.4). Also reported: sorted runs probed per lookup. (M3) Range-read cost = sorted runs opened per range lookup. (M4) Space amplification $\text{SA} = \text{on-disk bytes} / \text{live bytes}$. (M5) Read and write latency (mean, p95, p99) and stall time, *measured*, never derived from M1–M4 (Section 9), plus the longest wait of each level for the compaction slot (Theorem 5.9).

2 The experimental configuration and what it implies

Varied	$T \in \{2, 6, 10\}$ (plus 14, 20 in two cross- T cells); $k \in \{2, 4, 8, 16\}$; <code>max_bytes_for_level_base</code> $\in \{8, 16, 32\}$ MiB
Tree	<code>memtable</code> 2 MiB; SST 512 KiB; <code>num_levels</code> 13; <code>max_compaction_bytes</code> 12.5 MiB
Background	<code>max_background_jobs</code> 2; <code>max_subcompactions</code> 1; slowdown/stop 20/36 L0 files; pending limits 64/128 GiB
Reads	whole-key Bloom 10 bits/key; 4 KiB blocks; block cache 8 MiB; index/filter outside the cache; <code>open_files</code> 1000
Storage	compression none; paper mode: direct I/O on, WAL on (G5)
Workload	load 2.9M keys; 7.1M mixed ops, get/put/seek = 0.806/0.159/0.035; 1 client thread

Derived quantities used throughout. Loaded data $N \approx 2.9 \times 10^6 \times 1024 \text{ B} \approx 2,832 \text{ MiB}$ (2.77 GiB), i.e. about 5,660 SST files of 512 KiB. User bytes written in the measured phase $X \approx 7.1 \times 10^6 \times 0.159 \times 1024 \text{ B} \approx 1,102 \text{ MiB}$; per 50,000-operation mission about 7.8 MiB. With static targets the loaded tree has these shapes:

- $T=10$, base 16 MiB: L1 16, L2 160, L3 1,600 MiB, and L4 $\approx 1,056 \text{ MiB}$, i.e. 6.6% of its 16,000 MiB target.
- $T=2$, base 8 MiB: L1–L8 = 8, 16, $\dots$, 1024 MiB (2,040 MiB in total), and L9 $\approx 792 \text{ MiB}$, i.e. 38.7% of its 2,048 MiB target.

Implications for the analysis.

- E1 *Scale.*** The amplification results below depend on ratios (T , how full each level is, file size relative to level size), not on absolute sizes. The scaled-down geometry is defensible if these ratios are stated; the memory ratio (cache vs. data) is not scale-free and matters for reads (E8).
- E2 *Background parallelism.*** **[VERIFIED: `DBImpl::GetBGJobLimits` [34]]** With `max_background_jobs` = 2 (RocksDB’s default [33]), flushes get $\max(1, \lfloor 2/4 \rfloor) = 1$ slot and compactions $\max(1, 2 - 1) = 1$; in addition, compactions are held to one slot unless RocksDB signals that compaction must speed up. So this configuration always has exactly one compaction slot, and raising the job count raises compaction concurrency only while a speed-up is signalled.
- E3 *One client thread*** makes the benchmark a closed loop: when writes stall the client waits, so stalls appear as lost throughput, and p99 is lower than an open-loop system with many clients would see.
- E4 *compaction_pri fixed.*** The controller influences which level compacts; the picker chooses which file, and that choice sets the density factor δ_j of Section 3.3. All results are conditional on `kMinOverlappingRatio`.
- E5 *open_files* = 1000 vs. about 5,660 SSTs.** Table readers are evicted; in paper mode each reopen reads the file’s tail (footer, index, filter) from the device. This is the $o_r \omega_r$ term of Theorem 3.2. RocksDB’s default (−1, keep all files open) removes it.
- E6 *No compression*** (RocksDB defaults to Snappy when available) changes bytes written and stored; state it.
- E7 *perf_level* = 1.** **[VERIFIED: `perf_level.h` [32]]** Value 1 is `kDisable` (also `db_bench`’s default). Per-level read counters need a level of at least 2 (`kEnableCount`) and a call to `PerfCon-`

`text::EnablePerLevelPerfContext()`; only then can the terms of Theorem 3.2 be measured per level.

- E8** *Block cache.* 8 MiB, against about 2,832 MiB of data; RocksDB’s default has been 32 MB since version 8.2 [32], the value the Vertiorizon paper also uses as the latest default [8]. In paper mode it is the only cache, so the miss probability m_r is close to 1 except for hot blocks.
- E9** *Benchmark order.* The controller resumes right after the bulk load, while the load backlog is still draining. Insert `waitforcompaction` between `filluniquerandom` and `reset-stats/rlresume`, identically in every arm (Section 10.1).
- E10** *L0 is also scored by bytes.* **[VERIFIED: `VersionStorageInfo::ComputeCompactionScore` [32]]** With static targets, L0’s score is $\max(n_0/k, B_0/C_1)$, where B_0 is the size of the L0 files not being compacted. With flush files of $f_0 \approx 2$ MiB, L0 becomes due at about 4 files (base 8 MiB), 8 files (base 16 MiB) or 16 files (base 32 MiB), whatever larger k is set. In the grid $k \in \{2, 4, 8, 16\} \times \text{base} \in \{8, 16, 32\}$, only 9 of the 12 pairs are distinct: $k = 8$ and 16 act as $k = 4$ at base 8 MiB, and $k = 16$ acts as $k = 8$ at base 16 MiB.
- E11** *Pending-bytes limits.* 64/128 GiB are `db.bench`’s defaults (RocksDB’s own hard limit defaults to 256 GiB [32, 35]). With about 2.8 GiB of data, normal estimates stay far below them, so L0-file and memtable stalls are the ones that bind in this configuration.

3 Preliminaries

3.1 Notation

Levels $0, \dots, L$; S_j is the actual size and C_j the target size of level j ; $r_j = S_{j+1}/S_j$; T the size ratio; $f_L = S_L/C_L \in (0, 1]$ how full the last level is; N live bytes; e the entry size (near constant by G4); p_r the Bloom false-positive rate of run r ; ϕ the user ingest rate (bytes/s) and ϕ_j the byte flow through level j ; f_0 the size of a flush file; n_0 the number of L0 files; k_{eff} the effective L0 trigger (E10).

3.2 RocksDB mechanics under static targets

- *Targets.* **[VERIFIED: `CalculateBaseBytes`, `advanced_options.h` [32]]** $C_1 = \text{max.bytes_for_level_base}$ and $C_{n+1} = C_n \cdot T \cdot a_n$, where the a_n are the entries of `max.bytes_for_level\_multiplier\_additional` (default all 1); these entries are honoured only when dynamic level bytes is off [28]. They are integers and can be changed at runtime through `SetOptions()` [33]; T itself (`max.bytes_for_level_multiplier`) is a real number. Because each target is built from the one above it, raising a_n scales level $n+1$ and every deeper level. Only dynamic mode guarantees that about 90% of data sits in the last level [28].
- *Scores.* **[VERIFIED: `ComputeCompactionScore` [32]]** Ln ($n \geq 1$): bytes not being compacted / C_n (compensated sizes, which equal file sizes when there are no deletes). The `total_downcompact_bytes` adjustment applies only in dynamic mode [28, 29]. L0: $\max(n_0/k, B_0/C_1)$ (E10).
- *Picking.* **[VERIFIED: `ComputeCompactionScore`, `LevelCompactionBuilder` [32]]** After every version change and every pick, RocksDB sorts the levels by score. When the compaction slot is free it tries them in decreasing score order and compacts the first level with score ≥ 1 whose inputs are free; files within the level are chosen by `compaction_pri` [28, 31]. L0→L1 takes the overlapping L0 files.

- *Stalls* [30]: L0 files reaching the slowdown (20) or stop (36) trigger; estimated pending compaction bytes above the soft or hard limit (here 64/128 GiB); unflushed memtables reaching `max_write_buffer_number` (2). **[VERIFIED: EstimateCompactionBytesNeeded [32, 35]]** When L0 is due, the estimate adds the sizes of L0 and L1; each level over its target then adds $(S'_n - C_n)(S_{n+1}/S'_n + 1)$, where S'_n includes bytes cascaded from the level above. The estimate uses the targets RocksDB scores against, so under G7 it uses $m_i C_i$: $m_i > 1$ lowers it and $m_i < 1$ raises it (Section 5.3).

3.3 Write cost

Moving b bytes from level j into level $j+1$ rewrites the overlapping part of level $j+1$ [1, 5, 6]; merge policies have also been analysed as data-structure dynamization [2, 3, 4]. Under partial compaction a file overlaps about r_j equally sized files below, and uneven key density within a level lowers the average: Mo et al., citing Lim et al., use about $(T + 1)/2$ bytes written per byte for a level transition [8, 12]. We write the cost per byte moved as

$$w_{j+1} = 1 + \delta_j r_j, \quad \delta_j \in (0, 1], \quad (1)$$

with δ_j a per-level parameter measured from compaction statistics (Corollary 5.19); the $(T + 1)/2$ figure corresponds to $\delta = (T - 1)/(2T) \approx \frac{1}{2}$, and δ_j depends on `compaction_pri` and the key distribution. A trivially moved file costs nothing, not $1 + \delta_j r_j$; a measured δ_j averages trivial moves in, but the leading 1 of (1) overcounts them. If $\eta_j \leq 1$ is the fraction of user bytes that move from level j to $j+1$ (below 1 when obsolete versions are dropped), and the flush writes at most one byte per user byte (overwrites inside a memtable are dropped at flush),

$$\text{WA}_c \leq 1 + \sum_{j=0}^{L-1} \eta_j w_{j+1}. \quad (2)$$

For L0→L1 with keys spread over the whole range, every L0 file overlaps all of L1, so a batch of k_{eff} flush files of size f_0 costs $1 + S_1/(k_{\text{eff}} f_0)$ per byte. The WAL writes each user byte once more (plus small record headers); no compaction decision changes that amount.

3.4 Read cost: the full point-lookup model

Definition 3.1 (Search order). A `Get` of key κ consults the memtables, then the L0 files from newest to oldest, then levels $1, \dots, L$, consulting at level $j \geq 1$ only the one file whose key range covers κ . Index the consulted runs $r = 1, \dots, R$ in this order. Let $J(\kappa)$ be the first place (memtable, a run, or none) holding any version of κ ; the `Get` returns that version. For run r let: $c_r(\kappa) \in \{0, 1\}$ indicate that κ lies in the file's key range (checked in memory); p_r be the filter's false-positive probability; $m_r(\kappa)$ the probability that the needed data block misses the block cache; o_r the probability that the file's table reader is not open, and ω_r the reads needed to open it.

Theorem 3.2 (Expected storage reads of a `Get`). **[PROVEN]** (*whole-key filters; index and filter held by the open table reader, i.e. `cache_index_and_filter_blocks=0`; one data block per probe*)

$$\mathbb{E}[B(\kappa)] = \sum_{r < J(\kappa)} c_r(\kappa) (o_r \omega_r + p_r m_r(\kappa)) + \mathbf{1}[J(\kappa) \text{ is a run}] (o_J \omega_J + m_J(\kappa)).$$

If o_r and m_r do not depend on where the key's newest version lives (and p_r is read as the average false-positive rate over looked-up keys), averaging over looked-up keys gives

$$\text{RA} = \sum_r \Pr(J > r, c_r = 1) (o_r \omega_r + p_r m_r) + \sum_r \Pr(J = r) (o_r \omega_r + m_r).$$

Proof. Linearity of expectation over the runs in search order. A run before J holds no version of κ ; it is consulted only if $c_r = 1$, which requires its table reader (open cost $o_r\omega_r$); its filter then passes only by a false positive, which costs one data-block fetch that misses the cache with probability m_r . The run at J must be opened if needed, and its filter always passes because Bloom filters have no false negatives, so its data block is fetched. Nothing after J is consulted. The averaged form follows by taking expectations over κ with o_r, m_r independent of J . $\square$

In plain terms. The old model (lookups for missing keys, no cache, all tables open) is the special case $\Pr(J=r) = 0, m_r = 1, o_r = 0$. For lookups that find their key, the dominant term is the one block fetched where the key lives, plus $p_r m_r$ expected extra fetches for each run r passed on the way.

Measuring every term. RocksDB’s statistics [32] give memtable and per-level **Get** hits (GET_HIT_L0, GET_HIT_L1, GET_HIT_L2_AND_UP) for $\Pr(J)$; Bloom counters (BLOOM_FILTER_USEFUL, BLOOM_FILTER_FULL_POSITIVE, BLOOM_FILTER_FULL_TRUE_POSITIVE) for p , via $\hat{p} = (\text{positive} - \text{true positive}) / (\text{positive} - \text{true positive} + \text{useful})$; data-block cache hits and misses for m ; and file opens (NO_FILE_OPENS, TABLE_OPEN_PREFETCH_TAIL_*) for o and ω . Levels 2 and deeper need the per-level perf context (bloom counts, block-cache hits and misses, keys returned; E7). These counters do not attribute table opens to point lookups (seeks open tables too) or to levels, so the controller optimises RA_d , the data-block terms $p_r m_r$ and m_J measured per level on the **Get** path, and reports opens per operation separately. Its inputs change the $o_r\omega_r$ terms mainly through the number of files, which the data size and the file size largely fix.

3.5 Space

Lemma 3.3 (Obsolete bytes live above the last level). **[PROVEN]** (*equal entry sizes, G4; no snapshots, so a level ≥ 1 holds at most one version of a key; no deletes, G4*) Let $U = \sum_{j < L} S_j / S_L$, where the sum includes the memtables and L0. Then $SA \leq 1 + U$.

Proof. Map each obsolete version to the next newer version of the same key; the map is injective. The newer version lies in the memtables, in L0, or in a level above the obsolete one (a level ≥ 1 holds at most one version of a key), so it never lies in level L . With equal entry sizes, obsolete bytes $\leq \sum_{j < L} S_j$, so live bytes $\geq \text{total} - \sum_{j < L} S_j = S_L$ and $SA = \text{total}/\text{live} \leq \text{total}/S_L = 1 + U$. $\square$

With static targets and full upper levels, $\sum_{j=1}^{L-1} C_j = (C_L - C_1)/(T - 1)$, so $U \approx 1/(f_L(T - 1))$; with dynamic targets $f_L \approx 1$ and $U \approx 1/(T - 1)$. In the project’s trees $U \approx 1.68$ ($T=10$) and $U \approx 2.58$ ($T=2$), against 0.11 and 1 under dynamic targets. These values, and those of Table 1, leave out the memtables and L0 (at most 8 MiB), which add under 0.01. They are worst cases; the realised SA depends on how many overwrites have not yet been merged.

4 Requirement 1: compact / defer / defer+expand, with decay

4.1 MDP formalisation

Definition 4.1 (Trigger MDP). *State:* per level S_j, C_j , score, bytes in compaction, and an estimate of obsolete bytes; globally n_0 , pending bytes, memtables, busy compaction slots, and workload statistics per window. *Actions* (all realised through G7): at L0 only the trigger k (G3); at level $i \geq 1$, $a_i \in \{c, d, x(\varepsilon)\}$: *compact* (set m_i so that level i ’s score is at least 1, which compacts it early if $S_i < C_i$); *defer* (set m_i so that its score is below 1); *defer and expand* (set $m_i = 1 + \varepsilon$ with a

decay schedule). *Transition*: RocksDB itself, including its picker. *Reward*: minus a weighted cost of M1–M4 and stall time per window.

RocksDB does not expose obsolete bytes ahead of time; it reports dropped entries only after a compaction (per-level `KeyIn/KeyDrop` counts in the compaction statistics [32], which equal bytes up to the near-constant entry size). The state must therefore use an estimate. This is an open design item. G4 issues no deletes, so tombstones do not arise.

4.2 Space cost of temporary expansion

Theorem 4.2 (Space bound). **[PROVEN]** *Expanding level i to $(1+\varepsilon)C_i$ adds at most εC_i resident bytes plus the inputs of one running compaction (normally bounded by `max_compaction_bytes`, which RocksDB documents as a target it does not guarantee [32]). Relative to live data this is $\varepsilon C_i/N$, so a space headroom of B bytes allows $\sum_i \varepsilon_i C_i \leq B$ over the levels expanded at the same time; for one level, $\varepsilon_i \leq B/C_i$.*

Proof. Outflow from level i is enabled only above $(1+\varepsilon)C_i$, and a running compaction’s inputs persist until it finishes. Bounds for several levels add. $\square$

In plain terms. With a 2% example headroom ($B = 0.02N \approx 56.6$ MiB): at $T=10$, base 16 MiB, $\varepsilon \leq 3.5$ at L1, 0.35 at L2 and 0.035 at L3; at $T=2$, base 8 MiB, doubling ($\varepsilon = 1$) fits at L1–L3 only ($\varepsilon \leq 1.77$ at L3, 0.89 at L4, 0.055 at L8). Revision 1’s statement that doubling fits where $T^{L-i} \geq 50$ assumed dynamic targets and is withdrawn. The 2% figure is only an example. The project’s space bound is now the SLO box of Section 8.1: on-disk bytes at most $(1+\beta)S^*N$, where S^* is the knee’s space amplification, so the headroom is $B = (1+\beta)S^*N$ minus the current on-disk bytes, shared by the levels expanded at the same time.

4.3 Write effect: the generalised conservation identity

Assumption 4.3 (Fluid leveled model). Level i has fixed neighbours S_{i-1}, S_{i+1} and size $S_i(t) = s(t)S'_i$ with $s > 0$. Let $a = S'_i/S_{i-1}$ and $b = S_{i+1}/S'_i$ be the size ratios above and below it. A byte entering level i costs $\delta a s$ rewritten bytes; a byte leaving it costs $\delta b/s$ (equation (1) with a common δ). No obsolete versions are dropped, so $S'_i \dot{s} = \phi_{\text{in}} - \phi_{\text{out}}$. Data displaced by resizing level i ends up in the last level; for levels well above $L-1$ this changes S_L by a negligible fraction and is ignored here (Corollary 4.6 treats level $L-1$ exactly).

Theorem 4.4 (Generalised conservation identity). **[PROVEN]** *(Assumption 4.3) Over $[0, H]$, price every byte that enters or leaves level i under $s(t)$ at its actual rewrite cost and at the cost it would have if $s \equiv 1$. The difference is*

$$\Delta = \delta \left[\frac{aS'_i}{2} ((s(H) - 1)^2 - (s(0) - 1)^2) + (a - b) \int_0^H \phi_{\text{out}}(s - 1) dt + b \int_0^H \phi_{\text{out}} \frac{(s - 1)^2}{s} dt \right].$$

If $s(0) = s(H)$ (a cycle), Δ also equals the difference between the total rewrite costs of the policy $s(t)$ and the policy $s \equiv 1$ under the same inflow ϕ_{in} ; otherwise that difference is $\Delta - \delta b S'_i (s(H) - s(0))$, because the two policies then move different amounts of data out of level i .

Proof. Relative to $s = 1$, an entering byte pays $\delta a(s - 1)$ extra and a leaving byte pays $\delta b(1/s - 1) = -\delta b(s - 1)/s$. Hence $\Delta = \delta \int [\phi_{\text{in}} a(s - 1) - \phi_{\text{out}} b(s - 1)/s] dt$. Substitute $\phi_{\text{in}} = \phi_{\text{out}} + S'_i \dot{s}$: the term $\delta a S'_i \int \dot{s}(s - 1) dt$ integrates to the boundary term, and the remainder is $\delta \int \phi_{\text{out}}(s - 1)(a - b/s) dt$ with $(s - 1)(a - b/s) = (a - b)(s - 1) + b(s - 1)^2/s$. For the policy comparison, the policy $s \equiv 1$

with inflow ϕ_{in} moves ϕ_{in} out, so the difference in total costs is $\delta \int [\phi_{\text{in}} a(s-1) + \phi_{\text{out}} b/s - \phi_{\text{in}} b] dt = \Delta - \delta b \int (\phi_{\text{in}} - \phi_{\text{out}}) dt = \Delta - \delta b S'_i(s(H) - s(0))$. $\square$

Corollary 4.5 (Full interior levels: second order). **[PROVEN]** *If $a = b$ (both neighbours at target, static ratio T), any cycle with $s(0) = s(H) = 1$ has $\Delta = \delta T \int \phi_{\text{out}}(s-1)^2/s dt \geq 0$: expansion or early compaction and return never saves writes.*

Corollary 4.6 (The level above a partly filled last level: first order). **[PROVEN]** *(steady flow; no dropped versions; data conserved between levels $L-1$ and L) For level $L-1$ under static targets, let T be its ratio to the level above and $b = S_L/C_{L-1} = T f_L$. Holding level $L-1$ at a fraction s of its target moves the displaced data into the last level, so the two merge steps around it cost $\delta[Ts + (1+b)/s - 1]$ per byte passing through both, against $\delta(T+b)$ at $s = 1$. The minimum is at $s^* = \sqrt{(1+b)/T}$, with value $\delta(2\sqrt{T(1+b)} - 1)$. So when $b < T - 1$, compacting level $L-1$ earlier ($s < 1$) saves writes to first order and expanding it costs writes to first order.*

Proof. With $S_{L-1} = sC_{L-1}$ and $S_L = (1+b)C_{L-1} - sC_{L-1}$, a byte entering level $L-1$ rewrites δTs bytes and a byte leaving it rewrites $\delta S_L/S_{L-1} = \delta((1+b)/s - 1)$. Minimise $Ts + (1+b)/s$ over $s > 0$ (arithmetic-geometric mean inequality); the derivative at $s = 1$ is $\delta(T - 1 - b)$. $\square$

In plain terms. At $T=10$ ($b = 0.66$) the two steps cost 10.66δ per byte natively and 7.15δ at $s^* = 0.41$; at $T=2$ ($b = 0.77$) the difference is negligible (2.77δ at $s^* = 0.94$). `dynamic_level_bytes=true` removes the imbalance by fixing every ratio at T , which bounds space but is not the write-optimal fix (Table 1); a static configuration with a better-matched T and base size captures most of the gain. A controller's write gain must therefore be judged against the hull B3, with B2 reported alongside as a space reference.

Remark 4.7 (Static version). With $\prod_j r_j$ fixed, $\sum_j r_j$ is minimised by equal ratios (AM-GM), and perturbing one full interior level by $(1+\varepsilon)$ costs $\delta T \varepsilon^2/(1+\varepsilon)$. Unequal δ_j make the optimum non-geometric. Vertiorizon's choice $T' = T/\sqrt{2}$ for the ratio between a fully compacted upper part and the first partially compacted level is such a static choice: there the two merge steps have write costs T' and $(T^2/T' + 1)/2$, and AM-GM gives the optimum [8].

Remark 4.8 (Unequal density factors). **[PROVEN]** (same proof as Theorem 4.4) If merges into level i have density factor δ_{i-1} and merges out of it δ_i , Theorem 4.4 holds with a replaced by $\delta_{i-1}a$, b by $\delta_i b$, and the leading δ removed. A first-order term then appears even at a full interior level whenever $\delta_{i-1} \neq \delta_i$, with static optimum $s^* = \sqrt{\delta_i b / (\delta_{i-1} a)}$. Such a gain is again a fixed per-level target, i.e. a static choice, but static-mode RocksDB can express per-level multipliers only in integer steps (Section 3.2). Measure δ_j per level before relying on Corollary 4.5.

Theorem 4.9 (Elision bound). **[PROVEN]** *(fluid model with dropping; level sizes held at their targets) If deferral or expansion at level i drops an additional G bytes of obsolete versions inside level i , the write saving is at most $G W_{>i}$, where $W_{>i} = \sum_{j>i} w_j$, so the net change in writes is at least $\Delta - G W_{>i}$.*

Proof. A dropped byte avoids at most every later move $(w_{i+1}, \dots, w_L)$; with level sizes held at their targets the other bytes' costs are unchanged, and the surviving bytes obey Theorem 4.4. $\square$

In plain terms. Waiting pays only if new versions meet and cancel old ones before travelling down. The saving is first-order in the extra garbage dropped, while the geometry cost is second-order at full interior levels. Realised drops are measurable per level (`KeyDrop`); future drops are not.

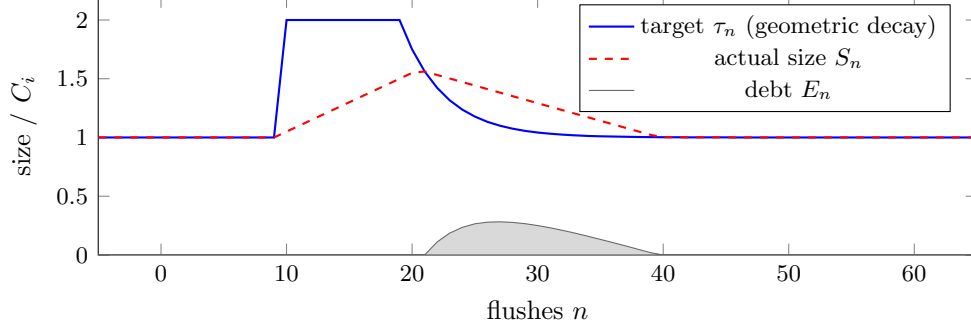


Figure 1: Expansion at $n = 10$ and decay from $n = 20$ (illustrative: inflow 0.05 and drain capacity 0.08 per flush, $\rho = 0.75$). Debt appears when the target falls faster than the level drains (Theorem 4.12).

Corollary 4.10 (Changing the level count). **[CONDITIONAL: stationary workload; cost rate at each moment that of the current geometry]** *If expansion lets the tree run with one level fewer, its long-run WA_c is at least that of the best fixed geometry with that level count, which also has the same run count for reads. Under static targets the level count is set by T and `max_bytes_for_level_base`, so `B3` must sweep the base size. Because the best fixed geometry is generally not geometric (Remarks 4.7–4.8, Section 5.5), a grid over T and base only approaches it from above (13.86 against 13.70 in Table 1).*

Proof. At each moment the tree has some fixed geometry with that level count, whose cost rate is at least the minimum over all such geometries. A time average of rates that are each at least this minimum is at least the minimum. $\square$

4.4 Read effect of expansion

Corollary 4.11 (Read change when keys move up one level). **[PROVEN]** (Theorem 3.2) *If expansion keeps the newest version of a looked-up key in level i instead of $i+1$, that lookup's expected storage reads fall by $p_i m_i + (m_{i+1} - m_i)$ (ignoring table opens). Range-read cost is unchanged.*

Proof. Before: the key is found at $i+1$, so run i contributes a false-positive probe $p_i m_i$ and run $i+1$ contributes m_{i+1} . After: run i contributes m_i and run $i+1$ is not consulted. The runs above i contribute the same in both cases. $\square$

In plain terms. Revision 1 claimed that expanding a level ≥ 1 leaves point reads unchanged; that was wrong. The false-positive part is about 1% of a block. The cache term can have either sign, because compaction creates new files whose blocks start uncached [18]. In paper mode, with an 8 MiB cache, $m \approx 1$ for cold data, so the change is close to p_i per affected lookup unless the moved keys are hot. Early compaction moves keys the other way and adds the same amount.

4.5 Decay schemes and stability

Let τ_n be the target after flush n , $E_n = \max(0, S_n - \tau_n)$ the compaction debt, and μ the spare drain capacity per flush interval.

Theorem 4.12 (Decay stability). **[PROVEN]** (per-flush fluid model; μ constant) *With $\Delta_n = \tau_{n-1} - \tau_n$, $E_n \leq \max(0, E_{n-1} + \Delta_n - \mu)$. (i) If $\Delta_n \leq \mu$ always, $E_n \leq E_0$. (ii) Geometric decay*

$\tau_n = C_i + \rho(\tau_{n-1} - C_i)$ converges as $\varepsilon C_i \rho^n$ and satisfies (i) iff $(1 - \rho)\varepsilon C_i \leq \mu$; a linear ramp over K flushes needs $K \geq \varepsilon C_i / \mu$. (iii) Snapping back at once ($\rho = 0$) creates debt εC_i in one step and raises the pending-bytes estimate by about $\varepsilon C_i(r_i + 1)$.

Proof. The target falls by Δ_n and the drain removes up to μ of excess; (i) by induction; (ii) $\Delta_n = (1 - \rho)\rho^{n-1}\varepsilon C_i$ is largest at $n = 1$; (iii) is $\Delta_1 = \varepsilon C_i$ with the estimate of Section 3.2. $\square$

In plain terms. Shrink a target no faster than compaction can drain the excess. With one compaction slot (E2), μ is shared by all levels, which couples this rule to Section 5.

Remark 4.13 (Native knob and the patch). `max_bytes_for_level_multiplier_additional` can express $x(\varepsilon)$ only as integer factors that also scale every deeper level, and cannot express $m_i < 1$ (Section 3.2). G7’s patch therefore either overrides one level’s target in the target computation (`CalculateBaseBytes`) or divides that level’s score by m_i in `ComputeCompactionScore`; RocksDB’s picker is untouched. The project’s implementation uses the second: targets stay native, so RocksDB’s pending-bytes estimate and its other uses of the targets are unchanged.

Verdict R1. Conditional. At full interior levels with equal density factors, expand-and-decay costs a small non-negative amount of writes and pays only through extra elision (Corollary 4.5, Theorem 4.9). At the level above a partly filled last level, the useful direction is the opposite (compact earlier), and most of that gain is reachable by a better static configuration, so it must beat the hull B3. A 2% example headroom allows doubling only at the top one to three levels (Theorem 4.2); the project’s headroom comes from the SLO box. *Falsified by:* expand-and-decay beating the hull B3 on WA_c in a workload with little garbage.

5 Requirement 2: per-level control, learned where the data allow

RusKey’s Lerp trains one DDPG actor–critic per level; each changes that level’s run bound K_i by -1 , 0 or $+1$ once per 50,000-operation mission, with reward $\alpha t_i + (1 - \alpha)t'$, $\alpha = \frac{1}{2}$ [9, 44]. Because deep levels give too little feedback, RusKey learns only Level 1 (uniform Bloom bits) or Levels 1–2 (Monkey’s Bloom allocation [5]) and derives deeper policies analytically (its *policy propagation*); its authors report that training all levels without this failed to reach the best policy from Level 3 down within 24 hours [9].

5.1 How much data a level- i controller needs

The number of *decisions* does not shrink with depth: under file-level compaction, and without dropped versions, each interior level passes on as many bytes as are written, one file at a time, so each level sees roughly the same number of compactions per ingested byte. What shrinks is how often a decision’s *consequence* can be observed. The two results below make this precise.

Lemma 5.1 (Data only moves down). **[PROVEN]** (no same-level compactions at levels ≥ 1 , as in this configuration: no deletes, and TTL and periodic compactions do not fire within a run) Compactions move entries from level j to $j+1$ (or $L0$ to $L1$), dropping some, and never upward. Hence (a) an entry version enters level $i \geq 1$ at most once; (b) the bytes entering level i during $(t_0, t]$ satisfy $V_i \leq A_i(t_0) + X(t_0, t]$, where $A_i(t_0)$ is the bytes in the memtables and levels $0..i-1$ at t_0 and X the user bytes written after t_0 .

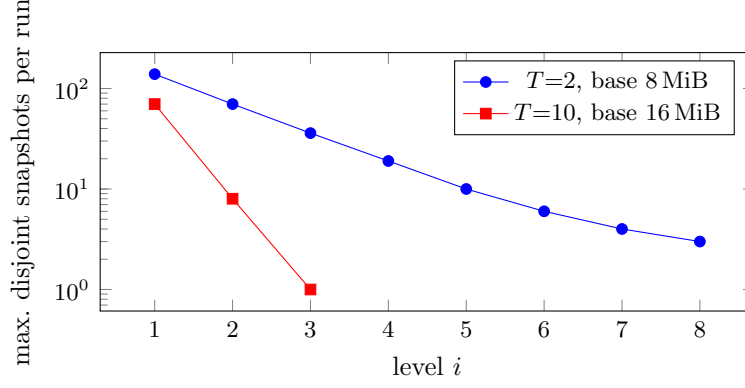


Figure 2: Theorem 5.2 for the project’s measured phase ($X \approx 1,102$ MiB; levels at target at the start; L0 holding 8 MiB; no dropped versions, which would only lower the bounds). At $T=10$ an L3 controller never sees a fully new L3 within a run.

Proof. (a) An entry that leaves level i is afterwards in a deeper level or dropped. (b) An entry that enters level i after t_0 was above level i at t_0 or was written after t_0 ; entries at or below level i at t_0 cannot enter it later. $\square$

Theorem 5.2 (Disjoint snapshots). **[PROVEN]** (*Lemma 5.1*) *Call two snapshots of level i disjoint if they share no entry. The number n of pairwise-disjoint snapshots observable during $(t_0, t]$ satisfies $n \leq 1 + (X + A_i)/S_{\min}$, where $S_{\min}$ is the smallest level- i size among them.*

Proof. Let consecutive disjoint snapshots be taken at $t_j < t_{j+1}$. An entry present at t_{j+1} entered after t_j : had it entered earlier, by Lemma 5.1(a) it would have stayed in level i through t_j , contradicting disjointness. So the inflow over $(t_j, t_{j+1}]$ is at least $S_i(t_{j+1}) \geq S_{\min}$. Summing over the $n - 1$ gaps gives $(n - 1)S_{\min} \leq X + A_i$. $\square$

In plain terms. To see a level in a genuinely new state you must push a whole level’s worth of data into it. With static targets and full interior levels, $S_i = C_1 T^{i-1}$, so the bound falls by a factor T per level.

Assumption 5.3 (Observation model for per-decision exploration). (N1) Over $[0, H]$ the reward signal is $y(t) = \mu + A u(t) + \xi(t)$ with $u(t) = \sum_k a_k g_i(t - t_k)$, decision times $t_k = k\Delta$ for $k = 0, 1, \dots$, and $a_k \in \{-1, +1\}$ the explored choice. (N2) ξ is Gaussian white noise with $\mathbb{E}[\xi(t)\xi(t')] = q\delta(t - t')$, the same for every level (the reward is a global signal); μ is known, which can only help the estimator. (N3) $g_i(s) = 1/D_i$ for $0 \leq s < D_i$ and 0 otherwise: each decision’s total effect A is the same at every level and is spread evenly over the level’s mean residence time D_i . (N4) The a_k are independent, ± 1 with probability $\frac{1}{2}$, and known to the learner.

Theorem 5.4 (Precision limit of per-decision exploration). **[PROVEN]** (*Assumption 5.3*) *Any estimator $\hat{A}$ that is unbiased for every exploration sequence satisfies $\text{Var}(\hat{A}) \geq q\Delta D_i/(H + \Delta)$. With Little’s law, $D_i = \bar{S}_i/\phi_i$ [36], so reaching variance v needs ingested data*

$$X = \phi H \geq \frac{\phi}{\phi_i} \cdot \frac{q\Delta \bar{S}_i}{v} - \phi\Delta \geq \frac{q\Delta \bar{S}_i}{v} - \phi\Delta.$$

Proof. Given the exploration sequence a , the model is a known signal $A u(t)$ in white Gaussian noise, whose Fisher information about A is $I(a) = q^{-1} \int_0^H u(t)^2 dt$ [37]. The Cramér–Rao bound

gives $\text{Var}(\hat{A} \mid a) \geq 1/I(a)$. By the law of total variance and Jensen’s inequality ($x \mapsto 1/x$ is convex), $\text{Var}(\hat{A}) \geq \mathbb{E}[q/\int u^2] \geq q/\mathbb{E}[\int u^2]$. Independence and zero mean of the a_k give $\mathbb{E}[u(t)^2] = \sum_k g_i(t - t_k)^2$. At most $H/\Delta + 1$ decisions fall in $[0, H)$, each contributing at most $\int g_i^2 = 1/D_i$, so $\mathbb{E} \int_0^H u^2 \leq (H/\Delta + 1)/D_i$ and the bound follows. Little’s law gives $\bar{S}_i = \phi_i D_i$ in steady state, and $\phi_i \leq \phi$ by Lemma 5.1. $\square$

In plain terms. Explored one decision at a time, each decision’s effect is smeared over the time the level takes to turn over, so it is buried under the same noise for T times longer at each deeper level. (Cramér–Rao is the best precision any unbiased estimate can reach; Fisher information measures how much a signal says about the unknown.)

Corollary 5.5 (Required data grows with level size). **[PROVEN]** *(under the respective assumptions) If the learner explores per decision, Theorem 5.4 makes its data need grow in proportion to $\bar{S}_i$. If it instead holds each choice long enough to see the level in a new state, Theorem 5.2 requires at least S_i of inflow per comparison. Either way the need grows by a factor T per full interior level.*

Remark 5.6 (What is and is not proven). This is a lower bound; the true need may be larger. Assumption (N3) is the crux: if a decision’s effect lands all at once after a delay, the dilution in Theorem 5.4 disappears and only the waiting time D_i remains, while Theorem 5.2 still holds.

5.2 Coupling between controllers

Lemma 5.7 (Coupling). **[PROVEN]** *Controller $i+1$ ’s transitions depend on π_i , since its inflow is level i ’s outflow, and its reward depends on π_i through the shared compaction slot and stalls. While π_i learns, controller $i+1$ ’s environment is non-stationary.*

The joint problem is a decentralised POMDP [53]. For that general class, deciding whether some finite-horizon joint policy reaches a given value is NEXP-complete for two or more agents [54]; this is a worst-case statement about the class, not about this particular problem. The convergence guarantees of single-agent Q-learning and actor–critic methods assume a stationary MDP [38], and independent learners in cooperative games can settle on poor joint policies or fail to converge [56, 55]. **[CONDITIONAL: no centralised critic]** Per-level independent RL has no convergence guarantee.

5.3 Safety envelope under RocksDB’s picker

Under G7 the controller and its supervisor act only through k and the multipliers m_i ; RocksDB compacts, whenever the slot is free, the highest-scoring level with score ≥ 1 (Section 3.2). There is one compaction slot (E2).

Definition 5.8 (Safety envelope through the scoring inputs). Each controlled level i has a nominal range $m_i \in [m_i^{\min}, m_i^{\max}]$ with $m_i^{\max} = 1 + \varepsilon_i^{\max}$ and $\varepsilon_i^{\max}$ set by the space cap (Theorem 4.2); the trigger has a range $k \in [k_{\min}, k_{\max}]$. A supervisor runs in the critical section in which RocksDB computes scores (Proposition 5.13) and overrides the controller by these rules, in order of precedence; each rule sets only the inputs that no higher rule has set:

- S1** *L0 guard.* If $n_0 \geq 20 - \mu_0$: set $k := k_{\min}$ and $m_i := m_i^{\max}$ for every controlled level.
- S2** *Debt guard.* If RocksDB’s pending-bytes estimate, or the same estimate recomputed with the native targets C_i , is at least θ times the soft limit: set $m_i := 1$ for every controlled level.
- S3** *Ageing* (optional). If controlled level i has had score ≥ 1 for a wait of τ_{age} without being picked: set $m_i := \min(m_i, S_i^{\text{nc}}/(\sigma_{\text{age}} C_i))$, so that its score is at least σ_{age} (S_i^{nc} : bytes not being compacted).

S4 Clamp. Otherwise k and every m_i are clamped to their nominal ranges.

Theorem 5.9 (What the envelope guarantees). **[PROVEN]** (one compaction slot, E2; RocksDB’s scoring and picking as verified in Section 3.2; every compaction lasts at least $\tau_{\min} > 0$)

- (i) Forced compaction and size bound. A controlled level i with $S_i^{\text{nc}} \geq (1 + \varepsilon_i^{\max})C_i$ has score ≥ 1 whatever the controller does, and exceeds $(1 + \varepsilon_i^{\max})C_i$ by at most the inflow it receives while it waits for the slot; each compaction into it adds at most that compaction’s input from the level above, normally bounded by `max_compaction_bytes`.
- (ii) No L0 slowdown. Let $\sigma_0 = (20 - \mu_0)/k_{\min}$. Suppose (a) μ_0 exceeds the number of flushes that can complete during the longest compaction of a level ≥ 1 plus one L0→L1 compaction, and (b) while S1 is active every level $j \geq 1$ has score below σ_0 (for a controlled level, $S_j^{\text{nc}} < \sigma_0(1 + \varepsilon_j^{\max})C_j$; for a native level, $S_j^{\text{nc}} < \sigma_0 C_j$). Then n_0 never reaches the slowdown trigger.
- (iii) Bounded wait with ageing. Suppose S3 is enabled with $\sigma_{\text{age}} < \sigma_0$, S2 is inactive, and while a controlled level i waits every other level $j \geq 1$ keeps score below σ_{age} . Then i is picked at the first release of the slot, after its wait reaches τ_{age} , at which L0’s score is also below σ_{age} .
- (iv) No deadlock; native fallback. Whenever the slot is free and some level has score ≥ 1 with free inputs, a compaction starts. While S2 is active the targets are native, so from then on the tree follows native RocksDB’s rules.

Proof. (i) In every rule $m_i \leq m_i^{\max}$ (S1 sets it to $m_i^{\max}$, S2 to 1, S3 lowers it, S4 clamps it), so the score $S_i^{\text{nc}}/(m_i C_i) \geq S_i^{\text{nc}}/((1 + \varepsilon_i^{\max})C_i) \geq 1$. Outflow from level i only needs the slot, and during the wait only compactations from the level above write into level i ; each adds at most its input from that level. (ii) Scores are recomputed at every version installation, including the one that installs each flush (Proposition 5.13), so S1 takes effect as soon as n_0 reaches $20 - \mu_0$. L0’s score is then at least $n_0/k_{\min} \geq \sigma_0$, above every other score by (b), so RocksDB picks L0 as soon as the running compaction ends, and the L0→L1 compaction then runs. By (a), fewer than μ_0 flushes complete in that time, so $n_0 \leq 19$. (iii) S1 is inactive at such a release, since while it is active $k = k_{\min}$ and L0’s score is at least $\sigma_0 > \sigma_{\text{age}}$; so S3 applies, level i ’s score is at least σ_{age} , every other score is smaller, and RocksDB compacts the highest-scoring level. (iv) The envelope changes only k and the targets. RocksDB’s picker, its tracking of files being compacted and its stall logic are unchanged, and the picker starts a compaction whenever the slot is free and a level with score ≥ 1 has free inputs. Under S2 every $m_i = 1$. $\square$

Remark 5.10 (What was given up, and why). Revision 2 used an ageing arbiter that chose among candidate levels itself. That bounded every level’s wait unconditionally, but it replaced RocksDB’s level selection, which G7 forbids, and it made early compaction impossible, because a level became a candidate only when its native score reached 1 and controllers could only allow or block it. (Its proof also assumed that a level’s new candidacy starts only when that level is picked, which fails once a controller can block and re-allow a level.) Native picking has no ageing term, so waits are now bounded only under Theorem 5.9(iii)’s condition; report the longest wait per level in every run (M5).

Remark 5.11 (The stall estimate under patched targets). RocksDB computes its pending-bytes estimate against the targets it scores with (Section 3.2). Raising m_i therefore hides debt from RocksDB’s stall logic and lowering m_i inflates it, which is why S2 checks the estimate with native targets as well. Under score scaling, the implementation chosen in Remark 4.13, targets stay native, so RocksDB’s own estimate is already the native one and S2’s second check is redundant but harmless. At this project’s scale neither estimate approaches the limits (E11).

In plain terms. With $k_{\min} = 2$ and $\mu_0 = 4$, $\sigma_0 = 8$: L0 is guaranteed the slot whenever no deeper level is more than eight times over its effective target. At base 16 MiB L0 becomes due at 8 files whatever larger k is set (E10), which leaves 12 flushes of margin; only at base 32 MiB does $k = 16$ take effect, and then only four flushes (8 MiB) of margin remain, so μ_0 must be chosen together with k and the base size. The number of flushes that complete during a compaction depends on device speed, so μ_0 is calibrated from paper-mode measurements (Section 9.2).

5.4 Decision staleness

Theorem 5.12 (Staleness of out-of-process decisions). **[CONDITIONAL: structural changes are Poisson with rate λ]** *A decision computed from a snapshot and applied after a round trip R is stale with probability $1 - e^{-\lambda R}$.*

Proof. $\Pr(\text{no change in } R) = e^{-\lambda R}$. □

Revision 1 inferred λ from $R \approx 50$ ms; that was an error, because 50 ms was the controller’s decision interval, not a measured round trip. The inference is withdrawn.

Proposition 5.13 (Zero staleness in process). **[VERIFIED: VersionSet::LogAndApply, Append-Version, DBImpl::BackgroundCompaction, LevelCompactionBuilder [32]]** *RocksDB installs each new version, and computes its compaction scores, with the DB mutex held; it picks compactions with the same mutex held, and recomputes the scores after each pick. (Only the target calculation of a not-yet-installed version runs without the mutex, and that version is invisible until installed.) Hence, if the controller’s multipliers and k are computed from the current version inside that critical section, the state they read is the state at which RocksDB applies them.*

Proof. A structural change is the installation of a new version or a change in the set of files being compacted; both happen under the same mutex, so none can occur between the read and the use. □

Option	Staleness	Cost / risk
1. Train in Python, infer in C++ (weights pushed at a fixed interval)	Zero for decisions (Prop. 5.13); weights up to one push interval old	Weight lag is tolerated by off-policy learners such as DQN, whose replay already mixes data from older policies; record the weight version with each transition. Transitions flow one way, so none is rejected. Define the push interval in operations, not seconds (Section 9.2).
2. Training and inference in C++	Zero	A C++ training stack, and training competes for CPU with one flush, one compaction and one client thread. No staleness benefit over option 1.
3. Controller as a .so plugin	Unchanged by itself	Packaging only: RocksDB has no public extension point for per-level fractional targets, so the target computation is still patched (Remark 4.13). Useful on top of option 1.

A RusKey-style loop that sets parameters once per mission (Section 13) is also free of staleness, because RocksDB applies them to the live tree.

5.5 Setting deep levels without controllers: a propagation law

Theorem 5.2 shows that controllers at deep levels cannot learn within a run. We ask whether deep-level targets can instead be *computed*, from quantities measured at run time and from the top-level decisions.

Relation to prior work. Setting deep levels from a model instead of learning them is the idea of RusKey’s *policy propagation* [9, §5.2], which learns the run bound of Level 1 (or Levels 1–2) and derives deeper ones from a cost analysis, under Monkey’s filter allocation by a three-level recurrence. Lim et al. already optimise per-level sizes numerically for skewed key-popularity distributions [12], and the LSM-bush uses per-level size ratios [7]. The derivation below is our own; its building blocks are the per-merge rewrite cost of Section 3.3 and the convexity of sums of exponentials. What it adds, subject to a fuller literature check, is: (a) what it sets (the level sizes of a static-target RocksDB tree, not run bounds); (b) a closed-form stationarity condition under a power-law rewrite-gap model; (c) inputs measurable from RocksDB’s statistics (Corollary 5.19); and (d) the neighbour-only coordination of Theorem 6.4. The paper must cite [9, 12] for the idea and the numerical precedent.

Assumption 5.14 (Propagation model). (P1) Levels $0, \dots, L$. $S_0 \approx k_{\text{eff}} f_0$ (set by the L0 trigger) is fixed, and so is the data D held in levels ≥ 1 ; the last level holds the rest, $S_L = D - \sum_{i=1}^{L-1} S_i$. The interior sizes are chosen, with $S_0 < S_1 < \dots < S_{L-1}$. Moving a byte from level j to $j+1$ costs $w_{j+1} = 1 + \delta_j \rho_{j+1}$ with $\rho_{j+1} = S_{j+1}/S_j$ (equation (1)). (P2) *Age bands*. Level $j < L$ holds the versions written between B_{j-1} and $B_j = S_0 + \dots + S_j$ bytes ago; the last level holds everything older. A level’s band width equals its size (little dead data inside a band). This is exact for whole-level merges and an approximation under file-level merges. (P3) *Rewrite gaps*. For a written version, let G be the number of bytes written (by all keys) before the same key is written again. Then $\Pr(G \geq x) = c(x/S_0)^{-\kappa}$ with $\kappa \geq 0$ for x between S_0 and S_{L-1} , where $c = \Pr(G \geq S_0) \leq 1$. (P4) Only write cost is optimised: resizing a level ≥ 1 moves point reads by about one false-positive probe per affected lookup (Corollary 4.11).

In plain terms. (P3) says that the chance a key waits more than x bytes for its next write falls off like $x^{-\kappa}$. A small κ means rewrites are spread over long gaps (little garbage meets its successor early); a large κ means most keys are rewritten soon. The constant c is the share of writes not rewritten within the L0 band; it multiplies every η_j below, so it scales W but cancels from the optimum. The numbers below take $c = 1$.

Lemma 5.15 (Where an old version dies). **[PROVEN]** (P2) *An old version whose key is rewritten g bytes later is dropped at the first level j with $S_j > g$. Hence the fraction of written bytes that move past level $j < L$ is $\eta_j = \Pr(G \geq S_j)$, which depends only on level j ’s own size.*

Proof. The new version enters level j when its age is B_{j-1} . At that moment the old version’s age is $B_{j-1} + g$, which lies inside level j ’s band $[B_{j-1}, B_j)$ exactly when $g < S_j$; the merge that brings the new version into level j then meets and drops the old one. Otherwise the old version is already deeper and the same test repeats at the next level. Sizes increase down to level $L-1$ and the last level’s band is unbounded, so a version moves past level $j < L$ exactly when $S_j \leq g$. $\square$

Theorem 5.16 (Optimal interior sizes for a given last-level size). **[PROVEN]** (Assumption 5.14, with S_L held fixed) *For a fixed last-level size S_L , the write cost per user byte, excluding the constant flush term,*

$$W(S_1, \dots, S_{L-1}) = \sum_{j=0}^{L-1} \eta_j (1 + \delta_j \rho_{j+1}),$$

has exactly one minimiser over positive sizes, and it is characterised by

$$\delta_{i-1} \rho_i^{1+\kappa} = (1 + \kappa) \delta_i \rho_{i+1} + \kappa, \quad i = 1, \dots, L-1. \quad (3)$$

Lemma 5.15, and hence the formula for W , requires sizes that increase with depth; when the minimiser has this property (as in every case checked below), it is the optimum of the model for that S_L .

Proof. Write $u_i = \ln S_i$. By (P3), $\eta_j = c \exp(-\kappa(u_j - \ln S_0))$, so every term of W is a positive constant times the exponential of a linear function of u ; such a sum is convex, and strictly convex here because the exponent directions span all u_i . Every sublevel set $\{W \leq C\}$ is bounded: on it, the ρ_1 term ($\propto e^{u_1}$) bounds u_1 from above, the ρ_L term ($\propto e^{-(1+\kappa)u_{L-1}}$) bounds u_{L-1} from below, and each term $\propto e^{u_{j+1} - (1+\kappa)u_j}$ carries the upper bound down the chain ($u_{j+1} \leq (1+\kappa)u_j + c'$) and the lower bound up it ($u_j \geq (u_{j+1} - c')/(1+\kappa)$). A continuous, strictly convex function whose sublevel sets are bounded has exactly one minimiser, where the gradient vanishes. Differentiating, $\partial W / \partial u_i = \eta_{i-1} \delta_{i-1} \rho_i - \kappa \eta_i (1 + \delta_i \rho_{i+1}) - \eta_i \delta_i \rho_{i+1}$. Setting this to zero, dividing by η_i and using $\eta_{i-1}/\eta_i = \rho_i^\kappa$ gives (3). $\square$

Corollary 5.17 (What cancels, and how to solve it). **[PROVEN]** (fixed S_L) Equation (3) contains only κ , the density factors and size ratios: the scale of the gap distribution, the absolute sizes and the ingest rate cancel. Given κ and the δ_j , choose ρ_1 , generate $\rho_2, \dots, \rho_L$ from (3), and adjust ρ_1 until $\rho_1 \cdots \rho_L = S_L/S_0$; bisection over the range of ρ_1 in which every ρ_j is positive finds the unique solution. With $\kappa = 0$ and equal δ_j the ratios are equal (plain rebalancing).

Proof. From (3), $\rho_{i+1} = (\delta_{i-1} \rho_i^{1+\kappa} - \kappa) / ((1+\kappa) \delta_i)$ increases with ρ_i , so every ρ_i and the product increase with ρ_1 ; the product tends to 0 at the lower end of that range, so the product constraint has one root, which by Theorem 5.16 is the minimiser. With $\kappa = 0$ and equal δ , (3) reads $\rho_i = \rho_{i+1}$. $\square$

Proposition 5.18 (Data conservation). **[PROVEN]** (Assumption 5.14) (i) Let $S_i^*(S_L)$ be the minimiser of Theorem 5.16. If $S_L + \sum_{i < L} S_i^*(S_L) < D$ at the smallest S_L for which that minimiser still has sizes increasing in depth (as in the example below), there is exactly one S_L with $S_L + \sum_{i < L} S_i^*(S_L) = D$, the conservation fixed point; bisection on S_L finds it. (ii) Any interior minimiser of W with $S_L = D - \sum_{i < L} S_i$ satisfies (3) with the term $(\eta_{L-1}/\eta_i) \delta_{L-1} S_i / S_{L-1}$ added to its right-hand side. The fixed point of (i) omits this term, so it is optimal for its own S_L but not exactly optimal under conservation.

Proof. (i) By Corollary 5.17, a larger S_L needs a larger product $\rho_1 \cdots \rho_L$, hence a larger ρ_1 , hence larger $\rho_2, \dots, \rho_L$ and larger $S_i^* = S_0 \rho_1 \cdots \rho_i$. So $g(S_L) = S_L + \sum_{i < L} S_i^*(S_L)$ is continuous and strictly increasing; it exceeds D at $S_L = D$ and, by hypothesis, lies below D at the smallest admissible S_L , so $g(S_L) = D$ has exactly one root. (ii) With $S_L = D - \sum_j e^{u_j}$, the only new dependence on u_i is through S_L in the last term $\eta_{L-1} (1 + \delta_{L-1} S_L / S_{L-1})$, whose derivative is $-\eta_{L-1} \delta_{L-1} S_i / S_{L-1}$; add it to the gradient in the proof of Theorem 5.16 and divide by η_i . A computer-algebra check confirms the result. $\square$

Under conservation, W also contains terms $-e^{u_j - (1+\kappa)u_{L-1}}$, so the convexity argument of Theorem 5.16 no longer applies and uniqueness of the conserved optimum is not proven. In the example below, two optimisers started from several points reach the same minimiser, and the fixed point of (i) is within 0.34% of it in W .

Corollary 5.19 (Both inputs are measurable). **[PROVEN]** (Lemma 5.15, (P3)) $\eta_{j-1}/\eta_j = \rho_j^\kappa$, so each level gives an estimate $\kappa_j = \ln(\eta_{j-1}/\eta_j) / \ln \rho_j$. **[VERIFIED: internal-stats.cc, compaction-job.cc [32]]** RocksDB's compaction statistics are kept per output level: in level j 's row, **Rn** is the bytes read from the level above, **Rnp1** the bytes read from level j itself, and **Moved** the bytes moved without rewriting. Hence η_{j-1} is **(Rn + Moved)** of row j per user byte, and δ_{j-1} is

Geometry (sizes of levels ≥ 1 , MiB)	levels	$\kappa = 0$	$\kappa = 0.2$	$\kappa = 0.5$	U
Native static targets, $T=10$, base 16 (16/160/1600/1056)	4	16.33	11.98	8.68	1.68
Best grid point: static $T=6$, base 16 (16/96/576/2144)	4	13.86	10.13	7.32	0.32
<code>dynamic_level_bytes=true</code> , $T=10$ (2.5/25.5/255/2549)	4	19.32	–	–	0.11
<code>dynamic_level_bytes=true</code> , $T=6$ (10.9/65.6/394/2362)	4	14.37	10.59	7.76	0.20
Law as deployed: fixed point of Prop. 5.18	4	13.75	9.94	7.05	0.26
Exact conserved optimum (numerical)	4	13.70	9.93	7.05	0.31

Table 1: Modelled write cost W per user byte (Assumption 5.14 with $c = 1$; same tree as Figure 3; the grid is the project’s $T \in \{2, 6, 10, 14, 20\}$, base $\in \{8, 16, 32\}$ MiB, and $T=6$, base 16 is its best point at every κ shown, whatever the level count). U is the worst-case space bound of Lemma 3.3, without memtables and L0, shown at $\kappa = 0$; at $\kappa = 0.2$ and 0.5 it is 0.24 and 0.20 for the fixed point and 0.27 and 0.21 for the exact optimum. The $T=10$ dynamic geometry has an L1 smaller than the L0 band, which breaks Lemma 5.15’s premise, so only $\kappa = 0$ is reported for it.

Rnp1/(Rn + Moved) of row j divided by ρ_j ; KeyIn/KeyDrop give a cross-check on dropped entries. If the κ_j agree across levels, (P3) holds for the workload; if they do not, the law’s premise fails. No learning is needed to set deep levels.

Remark 5.20 (Space budget). Adding a space price ν , i.e. minimising $W + \nu \sum_{i < L} S_i/N$ for fixed S_L , adds $\nu S_i/(N\eta_i)$ to the left-hand side of (3). The objective stays convex, and the price pushes deep levels, where S_i/η_i is largest, to be smaller.

Numerical check. We use the project’s $T=10$ tree: $S_0 = 8$ MiB, $D = 2,832$ MiB in levels ≥ 1 , native targets 16/160/1,600 MiB with 1,056 MiB left in L4, and $\delta_0 = 1$ for L0→L1 and $\delta_j = 0.5$ below. For fixed S_L , solving (3) by bisection agrees with direct and neighbour-only minimisation to 0.1 MiB (Table 2). Under conservation, two optimisers (Nelder–Mead and L-BFGS-B, several starting points) reach the same minimiser, and the conservation fixed point is within 0.34%, 0.14% and 0.03% of it in W at $\kappa = 0, 0.2$ and 0.5 (Figure 3, left). A computer-algebra check confirms the gradient, equation (3) and the conservation term; every number in Tables 1 and 2 was recomputed for this revision. Table 1 compares modelled write cost and the space bound U of Lemma 3.3 for the native targets, the best configuration in the project’s grid, `dynamic_level_bytes=true` at $T=10$ and $T=6$, the law as it would be deployed (the fixed point), and the exact conserved optimum.

What this means for R2 (within the model; see Section 5.6). (1) Deep levels need no controllers: their targets follow from measured κ and δ_j , the data size D , and S_0 , which the learned L0 trigger sets; recomputing them once per phase is what the law adds. (2) In the example, the law as deployed (the fixed point) writes 16% less than the native targets at $\kappa = 0$, but only 0.8% less than the best static configuration in the project’s grid (static $T=6$, base 16 MiB, which has the same number of levels), 1.9% less at $\kappa = 0.2$ and 3.7% less at $\kappa = 0.5$; the exact conserved optimum would add 0.3, 0.1 and under 0.1 percentage points. Knowing κ helps little: with fixed points, using the κ -aware geometry instead of the $\kappa = 0$ one changes modelled writes by -0.05% , -0.07% , $+0.5\%$ and $+3.3\%$ at $\kappa = 0.1, 0.2, 0.5$ and 1 (with exact optima: $+0.04\%$, $+0.17\%$, $+1.4\%$ and $+5.1\%$); at small κ the fixed point’s own error exceeds the gain. (3) So deep-level control is worth building only if validation finds heavy garbage reaching deep levels ($\kappa \gtrsim 1$, deletes, large overwrite volumes) or clearly unequal density factors (Remark 4.8); otherwise a well-chosen static configuration (B3) captures almost all of the modelled gain. (4) `dynamic_level_bytes=true` is not

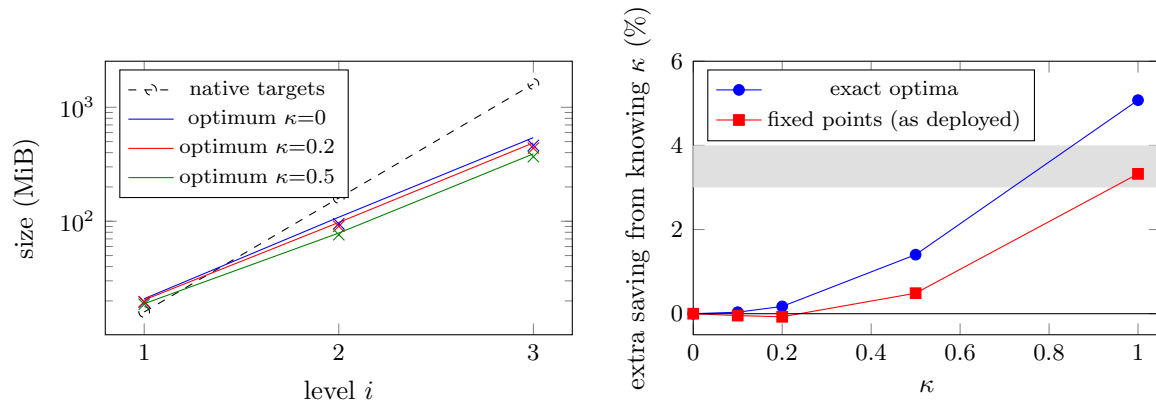


Figure 3: Numerical check of Section 5.5 on the $T=10$ tree, with data conserved. Left: interior sizes of the exact conserved optimum (lines; two optimisers agree) and of the conservation fixed point of Proposition 5.18 (crosses); native targets dashed. Right: extra modelled write saving from using the κ -aware geometry instead of the $\kappa = 0$ geometry, for exact optima and for the fixed points the law would deploy. The shaded band is the previous project’s run-to-run drift (3–4%), pre-restart context only (Section 11).

the write-optimal rebalancing: at $T=10$ it writes more than the native targets (19.3 vs. 16.3) but bounds space far better ($U = 0.11$ vs. 1.68).

5.6 Validation status and plan

Status. The results of this section hold inside Assumption 5.14. What has been checked is internal: the proofs; a computer-algebra check of the gradient of W and of equation (3) for general κ and per-level δ_j ; a computer-algebra check of the conservation term of Proposition 5.18; and the numerical agreement in Figure 3 and Tables 1 and 2. Uniqueness of the conserved optimum is checked numerically, not proven. None of this shows that RocksDB behaves like the model. The assumptions most at risk are (P2), because `kMinOverlappingRatio` picks the least-overlapping file rather than the oldest, so files of different ages share a level; (P3), which has never been measured; and (P1), because least-overlap picking and trivial moves above a nearly empty last level make δ_j depend on the tree’s state. Theorems 5.2 and 5.4 do not depend on this model.

Plan, cheapest first. Steps 1–3 form the paper’s measurement section for the law’s prerequisites, and step 6 its on/off comparison.

1. *Rewrite gaps, offline.* Log the key of every put from `db_bench` and compute each write’s gap to the same key’s next write. Plot the survival curve on log–log axes over the range from S_0 to S_{L-1} . This tests (P3) and gives κ and c , without a RocksDB run.
2. *κ , δ and garbage per level, from native runs.* Take κ_j , δ_j and the dropped bytes per level from the per-level compaction statistics (`Rn`, `Rnp1`, `Moved`, `KeyIn/KeyDrop`; Corollary 5.19). Check that the κ_j agree across levels and with step 1 (a joint test of (P2) and (P3)), and that δ_j is stable across time and seeds (P1).
3. *Direct test of Lemma 5.15.* Trace a sample of keys: record the gap at each write and the level where the old version is dropped, then compare with the predicted first level larger than the gap.

4. *Predictive test.* Run native, `dynamic_level_bytes=true`, and two or three patched per-level geometries. Compare measured compaction write amplification with the model's predictions. The model must rank them correctly, with prediction error smaller than the gains it claims.
5. *Sensitivity.* Check how far the optimum and the predicted savings move within the measured uncertainty in κ and δ .
6. *Propagation on vs. off.* Run the same top-level controller with deep levels set by the law (on) and left native (off), with paired seeds; report WA_c , RA, SA and latency. The difference is the law's measured contribution.

Decision rule. Use the propagation law (Theorem 5.16 at the fixed point of Proposition 5.18) and Scenario A of Section 6 only if steps 3 and 4 pass and step 6 shows a gain beyond run-to-run noise; otherwise follow Scenario B (deep levels native).

5.7 Which levels to learn: a test the learner can run

Theorem 5.2 bounds how many fresh states a level offers in a run. Whether that is enough depends on how large the effect to be found is compared with the noise, and the learner can check this itself.

Proposition 5.21 (Learnability test). **[PROVEN]** *(each setting of level i held for one turnover, called an epoch; per-epoch costs independent and Gaussian with a common standard deviation σ_i ; a fixed gap Δ_i between two settings' mean costs) With n epochs of each of two settings, the difference of their sample means has standard error $\sigma_i\sqrt{2/n}$. That difference exceeds z_α standard errors with probability at least $1/2$, i.e. a one-sided test at level α detects the gap at least half the time, iff*

$$n \geq \frac{2z_\alpha^2\sigma_i^2}{\Delta_i^2}, \quad \text{so the pair needs} \quad N_i = 4z_\alpha^2\left(\frac{\sigma_i}{\Delta_i}\right)^2$$

epochs in total. If at most a fraction f of the D_i epochs left in the run may be spent telling the settings apart, level i can be learned only if $N_i \leq fD_i$, that is, only if $\sigma_i/\Delta_i \leq \sqrt{fD_i}/(2z_\alpha)$.

Proof. The two sample means are independent Gaussians with variance σ_i^2/n each, so their difference has mean Δ_i and standard deviation $\sigma_i\sqrt{2/n}$. The test reports a gap when the difference exceeds $z_\alpha\sigma_i\sqrt{2/n}$. The difference is symmetric about Δ_i , so this happens with probability at least $1/2$ iff $\Delta_i \geq z_\alpha\sigma_i\sqrt{2/n}$, which rearranges to the bound on n . Two settings need $2n$ epochs, giving N_i ; requiring $N_i \leq fD_i$ and solving for σ_i/Δ_i gives the last condition. $\square$

In plain terms. (The standard error is the typical size of the error in an estimate; a one-sided test at level α wrongly reports a gap with probability α when there is none.) With $\alpha = 0.05$ ($z_\alpha = 1.645$) and half the remaining epochs kept for using what was learned ($f = \frac{1}{2}$), level i is learnable only if $\sigma_i/\Delta_i \leq \sqrt{D_i}/21.6$. Taking D_i from Figure 2:

	L1	L2	L3	L4	L5	L6
$T=10$: fresh states D_i	70	8	1			
largest learnable σ_i/Δ_i	1.80	0.61	0.22			
$T=2$: fresh states D_i	139	70	36	19	10	6
largest learnable σ_i/Δ_i	2.54	1.80	1.29	0.94	0.68	0.53

At $T=10$, L2 is learnable only if one setting beats another by at least 1.6 times the per-epoch noise; for a full interior level holding little garbage, Corollaries 4.5 and 4.11 predict far smaller effects, so

L2 stays native. At $T=2$, L2 and L3 qualify at moderate effects, and L4–L5 only with large ones. The test needs σ_i , which the learner measures while level i stays native, at no risk, and Δ_i , which the cost model predicts (Theorems 3.2, 4.4 and 4.9). Re-evaluated as the run proceeds, it lets a level whose effect grows (heavy garbage, Theorem 4.9) qualify, and it stops levels from qualifying near the end of a run. It is optimistic in the same way as Theorem 5.4: D_i is an upper bound, and correlated epochs, drift between phases and model error only raise N_i . It replaces a fixed per- T rule, which Revision 3 left open (O2).

Verdict R2. Feasible, restricted. Learn the L0 trigger and the levels that pass the learnability test (Proposition 5.21), which the learner runs itself: in the worked examples L1 at $T=10$, and L1–L3 at $T=2$ (L4–L5 only with large effects); keep deeper levels native (Scenario B of Section 6) or set them by Theorem 5.16 (Scenario A, only if Section 5.6 passes). Decisions are made in process (Proposition 5.13) within the envelope of Theorem 5.9. *Falsified by:* the learned levels failing to settle within the fresh states a run provides, or failing to beat B1 at equal read cost; for Scenario A, validation steps 3, 4 or 6 failing.

6 Requirement 3: what each controller observes

Definition 6.1. Controller i observes $o_i = (z_{i-1}, z_i, z_{i+1})$, where z_j is the size, target, score and bytes in compaction of level j .

Theorem 6.2 (Why global signals are needed). **[PROVEN]** (*counterexample*) *There are states $x \neq x'$ with $o_3(x) = o_3(x')$ whose optimal actions for controller 3 differ.*

Proof. Take one compaction slot (E2), flushes of $f_0 = 2$ MiB completing every τ_f , `max_bytes_for_level_base` = 32 MiB and $k = 16$, so that L0 becomes a compaction candidate exactly when $n_0 \geq 16$ (its score is $\max(n_0/16, 2n_0/32)$, E10). Use a reward that charges $c > 0$ per over-target byte per unit time and a penalty P for a slowdown. Let x and x' agree on L2–L4, with L3 over target and no obsolete versions in L3, so deferring L3 drops nothing (Theorem 4.9). Let the pending L3 compaction last D_3 and an L0→L1 compaction at most D_{01} , with $5\tau_f < D_3 < 6\tau_f$ and $D_{01} < 4\tau_f$. At time 0 a flush has just completed and the slot is free. In x , $n_0 = 0$. Compacting L3 now ends before n_0 reaches 16; the L0→L1 compaction that starts at 16 files ends before four more flushes complete, so $n_0 \leq 19$ and no slowdown occurs. Compacting now removes the over-target bytes sooner, so it is strictly better than deferring. In x' , $n_0 = 15$: L0 is not yet a candidate, so RocksDB starts the L3 compaction if controller 3 allows it. n_0 then reaches 20 after five flushes, before the L3 compaction ends, and the slowdown incurs P . If controller 3 defers, L0 becomes due after one flush, its compaction ends before four more flushes complete, and no slowdown occurs. For P larger than the over-target cost of the extra wait, deferring is strictly better. Since $o_3(x) = o_3(x')$, no policy of o_3 alone is optimal in both. $\square$

The same construction works with the pending-bytes limit (a sum over all levels), with cascades ($Li \rightarrow Li+1$ overflowing $Li+2$) and with space (dominated by the largest levels). Keeping L0 native (G3) does not change this, since n_0 remains a global state variable. The global view resolves it: with μ_0 chosen by Theorem 5.9(ii)(a) (here $\mu_0 \geq 9$, since up to 5 flushes complete during D_3 and up to 3 during D_{01}), the L0 guard S1 fires in x' and defers L3.

Proposition 6.3 (When a local view suffices). **[CONDITIONAL: factored transitions; rewards additive over neighbourhoods; slack shared resources]** *Under these conditions o_i suffices for i 's best response (a factored decentralised POMDP [53]). The conditions fail near stalls, which is where decisions matter.*

Case (fixed S_L)	Neighbour-only updates (+ price)	Joint optimum
$\kappa = 0$, $S_L = 2,255$ MiB	19.5 / 95.0 / 462.7 MiB	same (from (3))
$\kappa = 0.5$, $S_L = 2,368$ MiB	18.7 / 76.2 / 368.7 MiB	same
$\kappa = 0.5$, $S_L = 2,368$ MiB, $\sum_{i < L} S_i \leq 400$ MiB	$\nu = 1.94$: 18.0 / 69.0 / 313.1 MiB, $W = 7.0720$	18.0 / 69.0 / 313.1 MiB, $W = 7.0720$

Table 2: Numerical check of Theorem 6.4 on the $T=10$ tree of Figure 3, with S_L fixed at the conservation fixed point for each κ (Proposition 5.18): cyclic best responses starting from native targets, and bisection on ν (with $N = D$), against the recursion (3) and a joint constrained optimisation (SLSQP). Sizes are L1/L2/L3; W excludes the price term. All values recomputed for this revision.

6.1 Two kinds of coupling

Theorem 6.2 fails the three-level view for two different reasons, which need different fixes. (i) *Transient coupling*: stalls, the single compaction slot and L0 pressure. This is where the counterexample lives; no local view can see it, so it must be handled centrally. (ii) *Steady-state coupling*: each level’s best size depends on its neighbours and on shared budgets such as space. Here, as shown below, the local view is enough once a broadcast price and two slowly changing global sizes are added, provided the propagation model of Section 5.5 holds. Because that model is not yet validated (Section 5.6), we give the mitigation for both outcomes.

6.2 Scenario A: the propagation model is validated

Theorem 6.4 (Local updates plus one price reach the joint optimum). **[PROVEN]** (*Assumption 5.14 with S_L held fixed; minimiser with sizes increasing in depth*) Fix the last-level size S_L and let $W_\nu = W + \nu \sum_{i < L} S_i / N$ with $\nu \geq 0$. (i) Level i ’s best size given all others depends only on S_{i-1} , S_{i+1} , the measured constants $\kappa, \delta_{i-1}, \delta_i$, and the broadcast values ν , S_0 and N (the last two matter only when $\nu > 0$); that is, on the three-level view plus measured constants and a few global numbers. It is unique. (ii) If levels repeatedly set their own size to this best response, in any fixed cyclic order, the sizes converge to the unique minimiser of W_ν . (iii) For a space budget $\sum_{i < L} S_i \leq B$ that some sizes satisfy strictly, there is a price $\nu^* \geq 0$ at which the limit in (ii) is the budget-constrained optimum; ν^* can be found by bisection, because total size never rises as ν rises.

Proof. (i) In $u = \ln S$, the only terms of W_ν containing u_i are $\eta_{i-1} \delta_{i-1} \rho_i \propto e^{u_i}$, $\eta_i \propto e^{-\kappa u_i}$, $\eta_i \delta_i \rho_{i+1} \propto e^{u_{i+1} - (1+\kappa)u_i}$ and $\nu e^{u_i} / N$, whose coefficients involve only S_{i-1} , S_{i+1} and the listed constants (S_0 enters through $\eta_j = c(S_j/S_0)^{-\kappa}$ as a common factor of the first three terms, so it cancels when $\nu = 0$). As a function of u_i alone this is strictly convex and grows without bound in both directions, so the best response exists and is unique. (ii) W_ν is continuously differentiable and convex with bounded sublevel sets (proof of Theorem 5.16; the added term keeps both properties), and each one-level minimisation has a unique solution. By the block-coordinate-descent theorem [47, Prop. 2.7.1], every limit point of the iterates is a stationary point, hence the unique minimiser. The iterates stay in the initial sublevel set, which is bounded, so the whole sequence converges. (iii) In u , minimising W subject to $\sum_{i < L} e^{u_i} \leq B$ is a convex problem with a strictly feasible point, so strong duality holds and a multiplier $\nu^*/N \geq 0$ exists whose Lagrangian minimiser solves the constrained problem [48, §5.2.3, §5.5]; that Lagrangian equals W_{ν^*} up to a constant, so (ii) applies. If x_1, x_2 minimise $W + \nu_1 h$ and $W + \nu_2 h$ with $\nu_1 < \nu_2$ and $h = \sum S_i$, adding the two optimality inequalities gives $(\nu_2 - \nu_1)(h(x_2) - h(x_1)) \leq 0$, so total size is non-increasing in ν . $\square$

In plain terms. Each level only needs to see its two neighbours, a few measured constants, and one number broadcast to everyone (a price on space), plus slowly changing global sizes (the L0 band, the last level and the live data). If levels take turns adjusting their own size, the tree settles at the same optimum a central planner would pick for that last-level size. With data conserved, an outer loop moves S_L to the fixed point of Proposition 5.18, which is near-optimal but not exactly optimal (within 0.34% in the example). This holds only inside the propagation model.

Remark 6.5 (Reaching the exact conserved optimum locally). The extra term of Proposition 5.18(ii), $(S_{L-1}/S_i)^{-\kappa} \delta_{L-1} S_i / S_{L-1}$, involves only S_i , S_{L-1} , κ and δ_{L-1} , so each level could include it given a broadcast of S_{L-1} . Convexity is then lost, so Theorem 6.4(ii) no longer guarantees convergence; in the example the gain over the fixed point is at most 0.34%.

Mitigations in Scenario A. (M1) *Global summary features* for every controller: L0 file count and distance to the slowdown trigger, pending bytes against the soft limit (with native and effective targets, Remark 5.11), whether the compaction slot is busy and with which level, immutable memtables, per-mission workload statistics, and space used against the budget. They are read in process under the same mutex, so they add no staleness (Proposition 5.13). (M2) *Safety supervisor*: the envelope of Definition 5.8 owns stalls and the slot and overrides local choices; in the counterexample’s state x' its L0 guard defers L3. (M3) *Prices for shared budgets*: space, and optionally compaction bandwidth, coordinated as in Theorem 6.4(iii). (M4) *Full view for the learners*: only the L0 trigger and L1 learn (Section 5), so they receive every global feature, or are merged into one agent.

Effects on the other requirements (Scenario A). *R2*: every level keeps its own decision rule, but deep levels use best responses rather than learning, so convergence is proven instead of depending on coupled learners. Physically re-sizing a level still takes about a level’s worth of inflow (Theorem 5.2) and must respect the decay rule (Theorem 4.12), so deep levels re-settle slowly after the top changes S_0 . *R1*: expansions are price-coordinated, so total extra space respects the budget automatically. *R4*: its genuine cases (idle I/O, draining L0 at a read phase) need global signals and belong to the supervisor and the top controller. *R5*: the SLO becomes the coordinator: read/write priority sets the reward weights, the balance box and space budget set the prices, and the per-phase selector updates both.

6.3 Scenario B: the model fails; an agent for the top levels, deep levels native

If validation fails, deep levels stay native and only the L0 trigger and the levels that pass the learnability test (Proposition 5.21) are controlled: in the worked examples L1 at $T=10$ and L1–L3 at $T=2$.

What goes away. There is no deep geometry to coordinate, so Theorem 6.4 is not needed. Space coupling becomes small: with a 2% example headroom ($0.02N \approx 56.6$ MiB), Theorem 4.2 caps expansion at $\varepsilon \leq 3.5$ for L1 (16 MiB) and $\varepsilon \leq 0.35$ for L2 (160 MiB) at $T=10$, so a cap derived from the SLO box’s space bound (Section 8.1) replaces the price.

What stays. The transient coupling: Theorem 6.2’s counterexample holds with the L2 controller in place of L3. With one compaction slot, an L2→L3 compaction started just before L0 becomes due can still cause a slowdown, and native deep compactations still compete for the slot and add to pending bytes. A pure three-level view remains insufficient.

Mitigations in Scenario B (all independent of the propagation model). (B-M1) *Global features*, as (M1), plus the scores of the native deep levels, since they compete for the slot. (B-M2) *Safety supervisor*: Theorem 5.9’s guarantees depend only on RocksDB’s scoring, picking and stall mechanics and on finite compaction times, not on (P1)–(P4). The native deep levels are picked by RocksDB as usual; condition (b) of Theorem 5.9(ii) bounds their scores. (B-M3) *One agent with one output per controlled level* instead of one agent per level: it sees the whole state and has a separate output for the L0 trigger and for each level that passes the learnability test (Proposition 5.21) [49]. This keeps per-level decisions but removes local views and agents chasing each other while they learn; with L1 and L2 the joint action is small (e.g. $4 \times 3 \times 3 = 36$). (B-M4) *Hard space cap* $\sum_i \varepsilon_i C_i \leq B$ over the controlled levels, with B the headroom of Theorem 4.2 taken from the SLO box, enforced by the clamp S4.

Effects on the other requirements (Scenario B). *R2*: Theorem 5.2 bounds the fresh states per level, and the bound depends on T (Figure 2): at $T=2$, L1–L5 each see at least 10 per run; at $T=10$, L1 sees about 70 and L2 about 8. Whether that is enough is decided during the run by the learnability test (Proposition 5.21); runs are cold (G6, O2). At $T=2$ with L1–L5 controlled, a flat joint action would have $4 \times 3^5 = 972$ values; the recommended architecture uses one output per level and moves each setting at most one grid step per decision (Section 13). *R1*: temporarily reshaping a full interior level with equal neighbour ratios costs only a second-order amount of writes (Corollary 4.5), so there gains must come from dropping obsolete versions or from timing. L1 is the exception: its upstream ratio is set by the L0 batch, $S_1/(k_{\text{eff}}f_0)$ (2 at base 16 MiB with $k_{\text{eff}} = 4$), not by T , so Theorem 4.4’s $(a - b)$ term makes resizing it a first-order, static choice; in the model of Table 1 at $\kappa = 0$, resizing L1 alone lowers W from 16.33 to 15.65 (Section 8.1). A static gain beats the hull B3 only if B3 has no configuration at least as good: a single-level multiplier lies outside B3’s grid, but in the worked example B3’s best point (13.86) is still better. **[VERIFY: Section 11 does not report where overwrites were absorbed in the earlier runs; add the per-level drop counts or remove this pointer]** If most overwrites are absorbed at L0–L2, that is exactly where the agent acts. *R4*: its real levers, the L0 trigger and idle-I/O timing, sit with the single agent. *R5*: the SLO sets per-phase reward weights and caps for that one agent.

Risk. With static targets and native deep levels, the imbalance above a partly filled last level remains (Corollary 4.6). A better-matched static configuration (B3) captures most of that write gain through levels the controller does not touch, and B2 (`dynamic_level_bytes=true`) has far lower space amplification (Table 1). The primary claim is therefore made against B1 (same geometry), for top-level and phase gains, with B3 alongside and B2 as a reference row; the paper states why static targets are a ground rule.

Verdict R3. Feasible with mitigation. Scenario A (propagation model validated): each level sees its neighbours, measured constants, one broadcast price and two global sizes, and the supervisor handles transients (Theorem 6.4). Scenario B (model fails): one agent with an output per controlled level (the L0 trigger and the levels that pass the learnability test), global features, the supervisor and a hard space cap, with deeper levels native. *Falsified by*: a controller without global signals matching one with them near stalls; for Scenario A, neighbour-only updates failing to reach the jointly optimised geometry in RocksDB.

7 Requirement 4: proactive vs. reactive triggering

Definition 7.1. *Reactive:* compact when the native score reaches 1. *Proactive:* may compact below it (early, $m_i < 1$) or defer above it ($m_i > 1$).

Theorem 7.2 (Where early compaction pays). **[PROVEN]** (*Assumption 4.3; uniform keys for (iii)*) (i) At a full interior level ($a = b = T$), holding occupancy $x < 1$ costs $\delta T(1 - x)^2/x$ extra per byte and changes neither the run count nor the range cost; point reads change only through Corollary 4.11’s per-lookup term (one extra false-positive probe, about 1% of a block, plus the cache term) for keys pushed down a level. It is dominated on writes. (ii) At level $L-1$ under static targets, compacting early saves writes to first order (Corollary 4.6); a static configuration with a better-matched T and base size captures most of this (Table 1). (iii) At $L0$ the only lever is the trigger k (G3), acting through k_{eff} : $L0 \rightarrow L1$ costs $1 + S_1/(k_{\text{eff}} f_0)$ per byte, while the average $L0$ file count, and with it the $L0$ part of Theorem 3.2, is roughly proportional to k_{eff} when compaction starts promptly.

Proof. (i) Theorem 4.4 with constant $s = x$ and $a = b$ (the boundary term vanishes): $ax + b/x - 2T = T(1 - x)^2/x$; the read change is Corollary 4.11 applied in reverse. (ii) Corollary 4.6. (iii) Every $L0$ file overlaps all of $L1$, so each $L0 \rightarrow L1$ compaction rewrites S_1 for $k_{\text{eff}} f_0$ incoming bytes; $L0$ then cycles between about 0 and k_{eff} files. $\square$

In plain terms. With base 16 MiB and 2 MiB flush files, $L0 \rightarrow L1$ costs $1 + 8/k_{\text{eff}}$: 5 at $k = 2$, 3 at $k = 4$, and 2 at both $k = 8$ and $k = 16$, because $L0$ ’s byte score makes it due at 8 files (E10). Lowering k buys fewer $L0$ files per lookup at a first-order write cost. That is the one trade the ground rules leave at $L0$, and it is the only place where compacting early removes runs from the read path; elsewhere it at most shifts reads by Corollary 4.11’s small per-lookup term.

Where being proactive genuinely helps. (a) *Moving work into idle I/O.* Let $\varphi(t) = \text{inflow} \cdot w - \mu(t)$ be the rate at which compaction debt grows, with drain capacity μ . Debt cannot go below zero, so it follows the one-sided reflection

$$D(t) = \max \left\{ D(t_0) + \int_{t_0}^t \varphi ds, \sup_{t_0 \leq u \leq t} \int_u^t \varphi ds \right\},$$

and no stall occurs iff $D(t) \leq D_{\text{max}}$ for all t **[PROVEN]** (deterministic flows). Draining early lowers only the first term: it helps only if a burst would otherwise find debt left over, and it costs the write premium of Theorem 7.2. SILK’s I/O scheduler, which uses low-load periods, prioritises low levels and preempts compactions, achieved up to two orders of magnitude lower p99 latency than RocksDB and TRIAD on a Nutanix production workload and synthetic benchmarks **[EMPIRICAL]** [14]; see also [13, 16, 15]. In paper mode compaction is device-bound, so this matters more than in iteration mode. (b) *At the start of a read-heavy phase*, once detected: lower k (Theorem 7.2(iii)). With few flushes the native trigger would leave up to $k_{\text{eff}} - 1$ $L0$ files from the previous write phase in place for the whole phase; each costs every `Get` a Bloom check (and a wasted block read with probability p) and every range scan one more run (Theorem 3.2). Lowering k drains them with one $L0 \rightarrow L1$ compaction. Acting *before* the phase requires predicting it. (c) *Purging tombstones* lowers space and point-read cost at a write cost [17]; it does not apply to G4’s workload, which issues no deletes, and matters only if deletes are added. Deferral helps under skew through elision (Theorem 4.9); Facebook’s RocksDB traces are strongly skewed [26]. **[VERIFY: the claim that the earlier runs absorbed overwrites at $L0$ – $L2$, so deep deferral dropped little, is not reported in Section 11]** It would be consistent with Theorem 4.9.

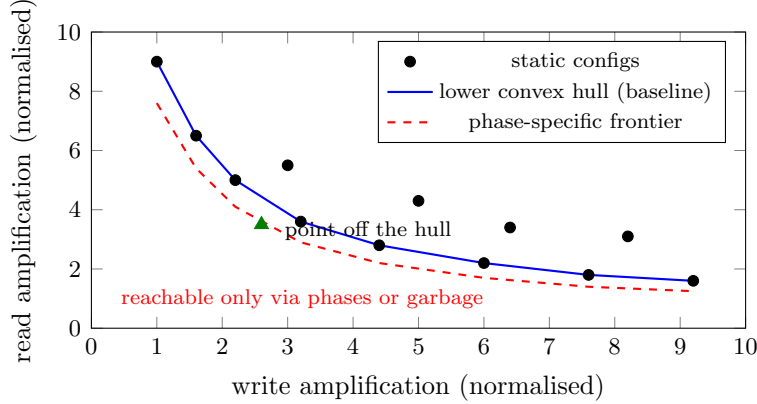


Figure 4: Static configurations, their convex hull, and where a dynamic policy can land (Theorem 7.3); illustrative.

7.1 Dynamic policies live in the convex hull

Theorem 7.3 (Hull theorem). **[CONDITIONAL: cost rate at each moment equals that of some static configuration; transition costs ≥ 0 ; stationary operation mix; all long-run averages taken per operation]** Let $F \subset \mathbb{R}^3$ be the (RA, WA_c, SA) points of static configurations. Every dynamic policy's long-run average lies in the closure of $\text{conv}(F) + \mathbb{R}_{\geq 0}^3$, so none strictly improves all three metrics over a Pareto-optimal point of $\text{conv}(F)$.

Proof. Measure time in operations. With a stationary mix, a fixed share of operations are lookups and each operation carries a fixed expected number of user bytes, so RA (per lookup), WA_c (per user byte) and SA (averaged over operations) are each the *same* weighted average of the static cost vectors, the weights being the fractions of operations run under each configuration, plus non-negative transition costs. If some point of that set were strictly below a Pareto-optimal point p of $\text{conv}(F)$ in all coordinates, its $\text{conv}(F)$ part would be too, contradicting Pareto-optimality. $\square$

In plain terms. A dynamic policy can beat a single static point merely by alternating between two static configurations, so the baseline must be the hull (B3). Weighting by operations matters: averaging each metric over its own clock (lookups, user bytes, seconds) with configurations that run at different speeds could produce a point outside the hull. The result is related to the RUM conjecture [25], which holds that read, update and memory overheads cannot all be optimal at once; the hull theorem adds that switching among static configurations does not escape their joint frontier.

Verdict R4. Conditional. Worth it only for moving work into idle I/O and lowering the L0 trigger at the start of read-heavy phases (tombstone purging would join them only if deletes are added); the write gain at level $L-1$ must beat the hull B3. *Falsified by:* early compaction at a full interior level lowering long-run WA_c or RA in a workload with little garbage.

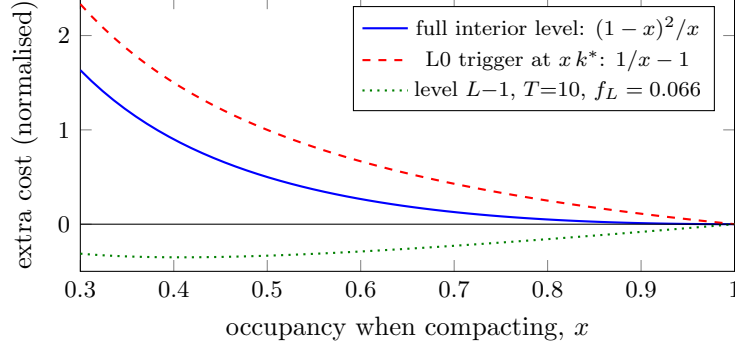


Figure 5: Write cost of compacting early (Theorem 7.2). The interior and $L-1$ curves are normalised by δT ; the L0 curve by its nominal L0→L1 cost $S_1/(k^* f_0)$. The $L-1$ curve (Corollary 4.6, with data conserved) is the only one below zero; most of that gain is also reachable by a better static configuration (Table 1).

8 Requirement 5: user-facing SLO with phase detection

8.1 Formalisation

The SLO is the project’s objective (decision O1). Let (R^*, W^*, S^*) be the *reachable knee*: among the static settings of the controller’s own inputs on the run’s geometry (k and the multipliers of the levels it controls, plus the propagation law’s geometry in Scenario A), measured in paper mode for the phase’s operation mix, the one minimising $R/R_{\min} + W/W_{\min} + S/S_{\min}$, with R measured as RA_d , W as WA_c and S as SA (M1, M2, M4). A priority $\pi \in \{\text{read}, \text{write}\}$ and a balance slack $\beta > 0$ give

$$\min_{\theta} M_{\pi}(\theta) \quad \text{s.t.} \quad M_j(\theta) \leq (1 + \beta)M_j^* \quad \forall j \neq \pi, \quad \text{stall time} \leq s_{\max}.$$

A mixed phase gets the priority *balanced*: minimise $M_R/R^* + M_W/W^*$ subject to $M_S \leq (1 + \beta)S^*$ and the stall bound. *Balanced, not extreme* is this box around the knee, solved with the detected phase’s cost model. As a constrained MDP, optimal stationary policies may randomise among a few deterministic ones [50], which is the alternation of Theorem 7.3. Lagrangian [52] and trust-region [51] methods learn such policies at a sample cost. Cost-model tuners already solve the static version [6, 7, 21, 22, 20, 23, 24]; ArceKV decides compactions and write stalls online from the observed workload [10].

Why the reachable knee, not the hull’s. Under G2 and G7 the controller cannot change T or the base size, so the knee of the hull B3 can lie outside the settings it reaches, and a box around it can then be infeasible; its constraint price would grow without bound. In the model of Table 1 at $\kappa = 0$ (L0 band 8 MiB, last level absorbing any change), native $T=10$ writes 16.33 per user byte and B3’s best point ($T=6$, base 16) 13.86. Resizing L1 alone reaches 15.65 (at $m_1 \approx 1.58$), and L1 with L2 reaches 15.36, so the write side of a B3-centred box needs $\beta \geq 0.11$ –0.13. With a 16 MiB L0 band ($k_{\text{eff}} = 8$) the figures are 13.80 and 13.06 against 12.86 for B3’s point, i.e. 7.3% and 1.5% apart. Only resizing the deep levels closes the gap: the law’s fixed point, reachable with $m \approx (1.22, 0.59, 0.29)$, writes 13.75. These are write-only comparisons, so they show the mechanism rather than locate the knee; but they show that the feasibility of a B3-centred box would hinge on β , on which levels are controlled and on the L0 band. A knee that is itself a reachable static

setting makes the box feasible by construction, up to measurement noise. Claims are still measured against B1 and B3 (Section 1).

Decision O1. The SLO box governs the headline claim. Revision 3’s alternative, a fixed $WA_c \leq 1.02 WA_c^{\text{native}}$ in every phase, is dropped; B1 remains a comparison baseline.

8.2 Phase detection

Proposition 8.1 (Detection delay). **[CONDITIONAL: independent operation types within a phase; known pre- and post-change distributions]** *CUSUM [57] with average run length to false alarm γ has worst-case delay $\approx \ln \gamma / D_{\text{KL}}(P_1 \| P_0)$, which is asymptotically optimal [58, 59]. For a write fraction moving from 0.1 to 0.9, $D = 0.8 \ln 9 = 1.76$ nats, so $\gamma = 10^6$ gives about 8 operations; from 0.5 to 0.7, $D = 0.082$ and about 168 operations.*

In plain terms. Noticing a phase change is cheap; acting on it (re-learning, restructuring the tree) is expensive. Real operation streams are correlated and the distributions must be estimated, so detect on short windows of operations with the noise measured, not assumed. In a simulation of such a detector (1,000-operation windows, multinomial operation counts, $\gamma = 10^6$, smallest shift of interest 5 points of put share, noise estimated from 20 windows), a 5-point shift from the base mix was flagged within 7 windows in all 500 trials (2.3 on average), a jump from 90% to 10% puts within the first window in all 200 trials, and 500,000 unchanged windows at the base mix raised no alarm. (CUSUM is a running sum of evidence for a change that raises an alarm when the sum crosses a threshold.)

8.3 Break-even phase length

Theorem 8.2 (Break-even). **[PROVEN]** *(additive cost model) For a phase of ℓ operations with per-operation gain g over the best single static configuration, detection delay d , re-convergence length K with excess cost δ_K per operation, and transition cost X_{tr} , switching pays iff*

$$g(\ell - d - K) > X_{\text{tr}} + \delta_K K \iff \ell > \ell^* = d + K + \frac{X_{\text{tr}} + \delta_K K}{g}.$$

Proof. Gains accrue only after detection and convergence; the losses are the transition and the learning excess. $\square$

In plain terms. For learned RL, RusKey needed about 300 missions (read- or write-heavy) to 600 missions (balanced) of 50,000 operations, i.e. $K \approx 15\text{--}30\text{M}$ operations **[EMPIRICAL]** [9], so phases must be much longer than that. A selector over a precomputed frontier has K near zero. All lengths are in operations or bytes, not seconds (Section 9.2).

Verdict R5. Feasible restricted: per-phase priorities within a $(1+\beta)$ box around the reachable knee, with latency measured. **Infeasible:** hard p95/p99 guarantees (Section 9), and RL that re-converges within each phase unless phases exceed ℓ^* . *Falsified by:* an RL SLO beating a per-phase selector over precomputed static settings (the reachable ones, which contain the knee) on phases shorter than 15M operations.

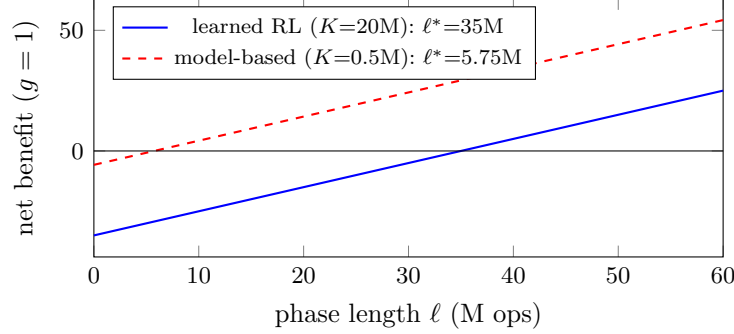


Figure 6: Break-even phase length (Theorem 8.2) with illustrative parameters $d = 0$, $X_{\text{tr}} = 5$, $\delta_K = 0.5$, $g = 1$.

9 Latency, measurement modes and the WAL

9.1 Lower amplification does not imply lower latency

Proposition 9.1 (Read latency decomposition). **[CONDITIONAL: additive service-time model]** *If the mean latency of a **Get** is $c_{\text{mem}} + c_{\text{probe}}\mathbb{E}[P] + t_{\text{blk}}\text{RA} + \mathbb{E}[W]$, where P is the number of runs probed (filter checks), t_{blk} the cost of one storage read (RA here is the full M2 count, table opens included) and W the time spent waiting behind background work, then a change that lowers RA by ΔR lowers mean latency iff $c_{\text{probe}}\Delta\mathbb{E}[P] + \Delta\mathbb{E}[W] < t_{\text{blk}}\Delta R$.*

In plain terms. In iteration mode a block read is a memory copy and the per-probe CPU can dominate; on fast storage, the CPU cost of Bloom-filter hashing can exceed the I/O cost [19]. A configuration with fewer block misses but more L0 files can therefore be slower. In paper mode t_{blk} is a device read and dominates, so RA tracks read latency much better.

Proposition 9.2 (Write latency). **[CONDITIONAL: additive service-time model]** *With the WAL on and sync off, a **Put**'s latency is the WAL append plus the memtable insert plus any stall delay. Write amplification is background work: it affects **Put** latency only through stalls and device contention. With the WAL off (iteration mode) the WAL term is absent.*

Proposition 9.3 (Tails are not a function of amplification). **[CONDITIONAL: stylised fluid schedules with level sizes at their targets, so a merge's cost does not depend on when it runs; closed-loop client, E3]** *There are two schedules that perform the same merges, and so have the same WA_c , whose SA agrees at the end of each period and whose RA differs by at most $f(20 - k_{\text{eff}})\bar{p}$, where f is the fraction of **Gets** issued while L0 holds extra files and $\bar{p}$ the mean filter false-positive rate; yet the p99 of **Put** latency of one exceeds the other's by the slowdown delay.*

Proof. Schedule A runs each deep merge as soon as it is due, below the drain capacity. Schedule B holds the same merges and runs them back-to-back once per period, starting as a flush burst arrives, so that L0 reaches the slowdown trigger while the slot is busy. The merges and their inputs are the same, so WA_c agrees, and the tree is the same at each period's end, so SA agrees there. During B's bursts L0 holds at most $20 - k_{\text{eff}}$ extra files, each adding one filter check to every **Get** and a wasted block read with probability $\bar{p}$, which bounds the RA difference. If at least 1% of **Puts** fall in B's slowdowns, B's p99 **Put** latency includes the slowdown delay and A's does not. $\square$

In plain terms. With $f = 2\%$, $k_{\text{eff}} = 4$ and $\bar{p} = 0.01$, the RA difference is at most about 0.003 blocks per **Get**, while the p99 of **Put** latency moves from microseconds to the slowdown delay.

Averages loosely follow amplification; p95 and p99 follow stall timing [14, 13]. Latency is therefore reported as a measured secondary result.

9.2 Paper mode vs. iteration mode

- *Same definitions.* The counters behind M1–M4 (compaction bytes, block-cache misses, file sizes) mean the same thing in both modes.
- *Different constants.* t_{blk} is a device read in paper mode and a memory copy in iteration mode; in paper mode $m_r \approx 1$ for cold blocks (E8) and table reopens hit the device (E5).
- *Different dynamics.* In paper mode flush and compaction run at device speed, compaction is busier, more L0 files accumulate and stall pressure changes. Trigger timing is what the controller learns, so a policy trained in iteration mode learned timing that does not exist in paper mode.

Consequences: one cost model, one reward and one learner design serve both modes, but training, evaluation and the static hull all use paper mode; iteration mode only debugs the harness. Every schedule (mission length, exploration decay, discount horizon, warm-up, and the interval at which weights are pushed to the C++ side) is defined in ingested bytes or operations, not seconds, so the design carries over unchanged and only the learned values differ. The previous controller’s schedules were in seconds (50 ms decisions, a per-second discount, 100 s cooling).

Which results depend on the mode. No proof changes: the results are stated in bytes, operations, files and cache-miss events, which mean the same in both modes. Three kinds of dependence remain. (1) *Reading.* RA counts the same events, but in iteration mode a miss is usually served by the operating system’s page cache, not the device, so Proposition 9.1’s conclusion flips. Proposition 9.2 assumes the WAL is on. Proposition 9.3’s slowdowns become rarer, and Section 7(a)’s idle-I/O gains shrink, because compaction no longer waits on the device. (2) *Conditions set by measured timing.* Theorem 5.9(ii)(a) requires μ_0 to exceed the flushes that complete during the longest compaction, which depends on flush and compaction speeds; Theorem 6.2’s construction, with long deep compactions, already needs $\mu_0 \geq 9$, more than the example $\mu_0 = 4$ after Theorem 5.9. So μ_0 is set from paper-mode measurements. Likewise Theorem 5.9(i)’s overshoot, Theorem 4.12’s drain capacity μ , Theorem 6.2’s timings and the noise q of Theorem 5.4 (and so the outcome of Proposition 5.21) take mode-specific values. (3) *Measured inputs.* The hull B3, the reachable knee, κ and δ_j (Corollary 5.19; δ_j depends on which files the picker chooses, and when), the re-convergence length K of Theorem 8.2, and every latency and stall figure must come from paper mode. Iteration mode remains useful for checking counters, plumbing and the phase detector, which counts operations.

9.3 Write amplification and the WAL

The WAL adds about 1 per user byte to every configuration. Including it would loosen the box’s write bound: with WAL term w , $\text{WA}_c + w \leq (1 + \beta)(W^* + w) \iff \text{WA}_c \leq (1 + \beta)W^* + \beta w$. The box is therefore stated on WA_c (M1), with WAL bytes reported separately. With the WAL off (iteration mode), $w = 0$ and WA_c is unchanged.

10 Learner choice: is a DQN enough?

Revision 1 did not adopt DDPG; it kept the previous project’s discrete actions, which is the DQN setting. Here we test whether a DQN is sufficient.

Proposition 10.1 (Discrete actions suffice). **[PROVEN]** (discounted MDP with factor $\gamma < 1$; bounded rewards) Let expansion factors lie in $[0, \varepsilon_{\max}]$ and suppose $Q^*(s, \varepsilon)$ is L_Q -Lipschitz in ε . The policy that is greedy with respect to Q^* over a grid of spacing h satisfies $V^*(s) - V^\pi(s) \leq L_Q h / (2(1 - \gamma))$ for all s .

Proof. Every ε is within $h/2$ of a grid point, so the grid’s best action is within $L_Q h/2$ of the continuous optimum. Since $Q^*(s, a) = r + \gamma \mathbb{E} V^*(s')$ and $V^\pi(s) = r + \gamma \mathbb{E} V^\pi(s')$ for $a = \pi(s)$, we get $V^*(s) - V^\pi(s) = [V^*(s) - Q^*(s, \pi(s))] + \gamma \mathbb{E}[V^*(s') - V^\pi(s')] \leq L_Q h/2 + \gamma \sup(V^* - V^\pi)$; taking the supremum gives the bound, in the manner of [41]. $\square$

In plain terms. Continuous actions, the reason DDPG exists, buy at most $L_Q h / (2(1 - \gamma))$ here; compact/defer is exactly discrete.

Proposition 10.2 (DDPG adds no convergence guarantee). **[PROVEN]** (by citation) Tabular Q -learning converges to Q^* under standard conditions [39]. With neural networks, DQN and DDPG both combine function approximation, bootstrapping and off-policy updates, a combination with no general convergence guarantee [38, 40].

Proposition 10.3 (Overestimation). **[PROVEN]** If two actions have equal true value m and their estimates carry independent $\mathcal{N}(0, \sigma^2)$ errors, $\mathbb{E}[\max(\hat{Q}_1, \hat{Q}_2)] = m + \sigma / \sqrt{\pi}$.

Proof. $\max(x_1, x_2) = \frac{1}{2}(x_1 + x_2) + \frac{1}{2}|x_1 - x_2|$ and $x_1 - x_2 \sim \mathcal{N}(0, 2\sigma^2)$, so $\mathbb{E}|x_1 - x_2| = 2\sigma / \sqrt{\pi}$. $\square$

In plain terms. With a value gap of 0.2 between compact and defer (the previous controller’s prior, pre-restart), estimate noise of $\sigma \approx 0.35$ already produces an upward bias ($\sigma / \sqrt{\pi}$) as large as that gap. Double DQN [43] removes most of it.

Conditions for a DQN to be enough. (1) A Markov state, i.e. the global features of Section 6. (2) A horizon covering the effect: per-level clocks, so that each decision is credited over the time its effect takes to play out (semi-Markov Q -learning [45, 46]). (3) Double DQN. (4) Exploration scaled to the value gaps and allowed only in safe states (Section 10.1). (5) Controllers only at levels that pass the learnability test (Proposition 5.21, built on Theorem 5.2).

Verdict. **A DQN is sufficient.** The failures seen so far (starvation at depth, exploration during the backlog, staleness) do not depend on DQN vs. DDPG.

10.1 Exploration at a cold start

Lemma 10.4 (Two-action softmax). **[PROVEN]** With value gap g and temperature τ , the non-greedy action is taken with probability $1/(1 + e^{g/\tau})$. Keeping it below p requires $\tau \leq g / \ln((1 - p)/p)$.

In plain terms. With $g = 0.2$ and $\tau = 1.0$ the defer probability is 0.450, as observed in the previous project; keeping it under 5% needs $\tau \leq 0.068$, which the previous schedule reached only near the end of its 100s cooling. Three fixes follow: (a) resume the controller only after `waitforcompaction` (E9); (b) anchor a cold agent on native RocksDB: take the native decision by default and deviate with small probability, only when there is no backlog and n_0 is below k_{eff} plus a margin; (c) keep levels that cannot turn over (Theorem 5.2) native.

11 Empirical context from the previous project

Pre-restart context only. The numbers below come from the controller the project abandoned in September 2026. They illustrate Theorems 5.2 and 4.9 and Lemma 10.4, but they are not evidence for the paper, and they were not re-checked for this revision (open decision O3 in Section 13).

[EMPIRICAL] (user’s measurements, in iteration mode: direct I/O off, WAL off; three repeats; Assoc/mixgraph; 10M operations; two-action DQN per level [42]; native picker). At $T=2$: +6.6% write and +3.0% read amplification against a same-seed native twin, dominated by static $T=6$ points. At $T=6$: −7.1% read for +9.7% write, a worse exchange rate than an analytic prior. At $T=10$: essentially native. At the trigger-2 comparators no static setting read less within +2% write. The audit’s step-by-step account at $T=2$: in the first 30 s after the controller woke, 3.5–5.1 GB of compaction was still owed from the load (0.07–0.33 GB normally), L0 held 9–14 files and level scores were 4–14; the controller deferred 43–45% of due deep levels; this added about 0.68 GB of writes per run, because deferred moves became real merges of unique loaded keys; the tree’s depth matched native; after 30 s the learner wrote 0.07 GB less than native.

12 Overall verdict

All verdicts are theoretical: each design below follows from the results cited, and each still awaits experimental confirmation.

Requirement	Verdict	Design	Key results		Falsified by
R1 compact / defer / expand, decay	Conditional	Expansion only at the top levels, within the space cap; targets fall no faster than the drain; gains only from dropped obsolete versions or timing	Cor. Thms. 4.9, 4.2, 4.12	4.5,	Expand-and-decay beating the hull B3 on WA_c with little garbage
R2 per-level control	Feasible, restricted	Learn the L0 trigger and the levels that pass the learnability test; other levels native (B) or set by the propagation law (A); all through k and m_i within the envelope	Thms. 5.2, 5.4, 5.16, 5.9; Props. 5.13, 5.21	5.2,	Learned levels not settling within a run, or not beating B1 at equal read cost; for A, validation steps 3, 4 or 6 failing
R3 observation	Feasible with mitigation	A: neighbours, broadcast price, supervisor. B: one agent with an output per controlled level, global features, supervisor, space cap	Thms. 6.2, 6.4, 5.9	6.4,	A controller without global signals matching one with them near stalls
R4 proactive triggering	Conditional	Only via the L0 trigger at read-phase starts and idle-I/O timing (tombstone purging only if deletes are added)	Thm. reflection condition of Section 7(a)	7.2;	Early compaction at a full interior level lowering long-run WA_c or RA
R5 SLO	Feasible, restricted	Per-phase priorities within a $(1 + \beta)$ box around the reachable knee; latency measured, not guaranteed	Thm. Prop. 8.1; Section 8.1	8.2;	An RL SLO beating a per-phase selector over static settings on phases shorter than 15M operations

13 Recommended architecture

1. *Scope.* Control the L0 trigger k (G3, acting through k_{eff}) and the multipliers of the levels that pass the learnability test (Proposition 5.21), which the learner evaluates during the run: in the worked examples L1 at $T=10$, and L1–L3 at $T=2$ (L4–L5 only with large effects). Keep all other levels native (Scenario B of Section 6, the default until validation), or, if Section 5.6’s tests pass, set them by the propagation law (Theorem 5.16 at the fixed point of Proposition 5.18) with price coordination (Scenario A), and only where Corollary 5.19 shows heavy deep garbage or unequal density factors. The controlled levels are driven by one agent with one output per level, each moving at most one grid step per decision. RocksDB picks every compaction (G7).
2. *Decisions without staleness.* The controller chooses k and the m_i in process, at mission boundaries, from parameters the learner pushes; RocksDB applies them at every rescore, which is exact (Proposition 5.13). Hold each level’s decision for about one turnover of that level: about 7.8 MiB is written per 50,000-operation mission, so at $T=10$ that is about 2 missions for L1 (16 MiB) and about 20 for L2 (160 MiB).
3. *Learner.* The one-step (contextual-bandit) form: a cost model built from Theorems 3.2, 4.4 and 4.9 with measured inputs, plus a small linear correction with uncertainty, learned per level and setting and scored on each held decision’s outcome. There is no bootstrapping, which avoids the combination that Proposition 10.2 warns about and Proposition 10.3’s bias. Native-anchored cold start (Section 10.1), all schedules in bytes or operations, controller resumed only after `waitforcompaction`. Every run starts cold (G6, O2).
4. *Safety.* The envelope of Definition 5.8, with μ_0 set from paper-mode measurements of the flushes completing during a compaction (Theorem 5.9(ii)(a), Section 9.2), together with k and the base size (E10); report the longest wait per level.
5. *Reward and constraint* on logical counters: WA_c , RA_d , runs probed, stall time; the SLO box of Section 8.1 around the reachable knee, per detected phase (O1); expansion within the box’s space bound (Theorem 4.2).
6. *Evaluation* in paper mode against B1 and the hull B3 (over distinct k_{eff} values, E10), with B2 as a reference row, many seeds; the reachable knee and the hull measured in paper mode; latency measured and reported as secondary; include the measurement section and the propagation on/off comparison of Section 5.6.

Decisions. (O1) *Decided:* the SLO box around the reachable knee governs the headline claim (Section 8.1). (O2) *Decided:* cold runs only; nothing learned is carried across runs, and the learnability test decides which levels learn (Proposition 5.21). (O3) *Open:* whether to keep the previous project’s numbers (Section 11, the band in Figure 3, Proposition 10.3’s example and Lemma 10.4’s “as observed”) as context or remove them.

14 Threats to validity

- *Density factors* δ_j vary with the picker and key distribution [12, 11]. A common factor only scales Theorems 4.4 and 7.2; unequal factors on a level’s two merge steps add a first-order term with a static optimum (Remark 4.8). Measure them per level.
- *Per-level cache behaviour* (m_r) is measured, not derived; it decides Corollary 4.11.

- *Obsolete bytes* are not observable before a compaction drops them; the elision term must be estimated.
- *Assumption (N3)* (effects spread over residence time) drives Theorem 5.4; Theorem 5.2 does not need it.
- *Age bands and power-law rewrite gaps* (P2, P3) drive Theorem 5.16. File-level merges only approximately preserve age order; the agreement of the per-level κ_j (Corollary 5.19) tests both. Under data conservation the optimum’s uniqueness is checked numerically, not proven (Proposition 5.18).
- *RocksDB version.* Every RocksDB detail marked **[VERIFIED:]** was checked on the `main` branch in September 2026 (L0 byte scoring, the pending-bytes formula, compaction concurrency, the `perf_level` values, mutex use, the picker’s order); re-check them on the version the project pins.
- *Changed compaction order.* With score scaling, RocksDB’s own stall estimate stays native (Remark 5.11), but the multipliers still change which level compacts when, and that shifts stall timing, which must be measured.
- *Closed-loop client and one compaction slot* (E2, E3) make results specific to this load level; report utilisation and add one higher-concurrency run.
- *Measurement mode.* Iteration-mode runs validate counters and plumbing only; μ_0 , the knee, the hull, κ , δ_j and the learner’s noise estimates must come from paper mode (Section 9.2).
- *Compression off and `open_files` = 1000* (E5, E6) change byte counts and add table-open reads.
- *Garbage-rich workloads* (deep overwrites or deletes) make Theorem 4.9’s saving large and could turn R1 feasible for writes; measure drops per level first. RocksDB’s evolving priorities are summarised in [27].
- *Novelty.* The closest prior work must be positioned explicitly: RusKey’s policy propagation for setting deep levels without learning [9]; Lim et al.’s numerical level-size optimisation under skew [12]; ArceKV, which decides online when to compact and when to stall, on RocksDB, under dynamic workloads [10]; and EcoTune, which treats compaction as an investment in future query throughput and chooses its timing and aggressiveness [24].

A Assumption register

#	Assumption	Used in	Status
1	Leveled compaction, file-level merges, <code>kMinOverlappingRatio</code>	all	ground rule (G1)
2	Static targets (<code>dynamic_level_bytes=false</code>)	§3.2, Thm. 4.4	ground rule (G2)
3	L0 native except trigger k ; effective trigger $\min(k, \lceil C_1/f_0 \rceil)$	§4, Thm. 7.2	ground rule (G3); byte scoring verified
4	Near-constant entry size; no deletes	Lemma 3.3, §7	ground rule (G4)
5	Paper mode: direct I/O on, WAL on, sync off	§9; all calibrations	ground rule (G5)
6	RocksDB picks every compaction; controller acts only through k and m_i	Def. 4.1, §5.3	ground rule (G7)

#	Assumption	Used in	Status
7	Overlap proportional to level size; per-level δ_j measured; a common δ on a level's two merge steps in Thm. 4.4	eq. (1), Thm. 4.4	model; see Remark 4.8
8	Fluid flows; neighbours fixed while level i changes	Thm. 4.4	simplification
9	L0 files overlap all of L1 (keys spread over the range)	Thm. 7.2(iii)	holds for uniform keys; skew weakens it
10	Whole-key filters; index/filter held by open table reader; one block per probe	Thm. 3.2	holds in this config
11	o_r and m_r independent of the key's location (averaged RA)	Thm. 3.2	conditional; per-key form exact
12	No same-level compactions at levels ≥ 1	Lemma 5.1	holds in this config
13	Global reward, white noise, effects spread over D_i , independent ± 1 exploration	Thm. 5.4	conditional ((N3) is the crux)
14	Stationarity for Little's law and Cor. 4.10	Thm. 5.4, Cor. 4.10	within a phase
15	Poisson structural changes	Thm. 5.12	motivation only
16	Versions installed, scores computed and compactions picked under one mutex	Prop. 5.13	verified (main, Sept. 2026)
17	One compaction slot	E2, Thms. 5.9, 6.2	verified for 2 background jobs
18	RocksDB scores L0 by files and bytes and picks the highest score ≥ 1	§3.2, Thm. 5.9	verified
19	Compactions last at least $\tau_{\min} > 0$; score-gap conditions (ii)(b) and (iii)	Thm. 5.9	conditional; check scores in runs
20	Constant spare drain per flush interval	Thm. 4.12	simplification
21	Deterministic flows for the stall condition	§7(a)	simplification
22	Quasi-static cost, stationary operation mix, per-operation averages	Thm. 7.3	conditional
23	Independent operations; known distributions	Prop. 8.1	conditional
24	Additive phase costs, constant per-operation gain	Thm. 8.2	simplification
25	Stylised fluid schedules	Prop. 9.3	construction
26	Discounted MDP, bounded rewards, Q^* Lipschitz in ε	Prop. 10.1	conditional
27	Obsolete bytes estimable for the state	§4	open
28	Numbers in Fig. 2: levels at target at start, L0 8 MiB, no drops	Fig. 2	stated for the example
29	Age-band ordering; band width equals level size (P2)	Lemma 5.15	approximation under file-level merges
30	Power-law rewrite-gap tail over level sizes (P3)	Thm. 5.16	testable via κ_j agreement
31	Deep-level resizing optimised for writes only (P4)	Thm. 5.16	justified by Cor. 4.11
32	Example δ (1 for L0→L1, 0.5 below) and $c = 1$	Fig. 3, Tab. 1	illustrative; measure per level
33	Data conserved: the last level holds what interior levels do not (P1)	Prop. 5.18, Tab. 1	holds by definition
34	Fixed S_L ; strictly feasible space budget; minimiser with sizes increasing in depth	Thm. 6.4	Scenario A only
35	Counterexample timings ($5\tau_f < D_3 < 6\tau_f$, $D_{01} < 4\tau_f$)	Thm. 6.2	chosen for the construction
36	Space headroom from the SLO box: $(1 + \beta)S^*N$ minus on-disk bytes	Thm. 4.2, R1	decided (O1); 2% kept as an example
37	Independent Gaussian per-epoch costs, common σ_i , fixed gap	Prop. 5.21	conditional; optimistic like Thm. 5.4

#	Assumption	Used in	Status
38	Reachable knee measured in paper mode per operation mix	§8.1	design rule (O1)

B Changes

B.1 Changes in Revision 4 (from Revision 3)

1. *Decisions O1 and O2 recorded.* The phase-aware SLO is the objective, and the fixed constraint $WA_c \leq 1.02 WA_c^{\text{native}}$ is dropped (Section 8.1); runs are cold and nothing learned crosses runs (G6). Sections 9.3 and 13 and the verdict table follow.
2. *Reachable knee* (Section 8.1). The SLO box is centred on the knee of the static settings the controller can reach on the run’s geometry, not on B3’s knee, which can make the box infeasible; the section gives the write-cost numbers behind this. A *balanced* priority is added for mixed phases.
3. *Learnability test* (new Section 5.7, Proposition 5.21). The learner decides which levels to learn by comparing the epochs needed to tell settings apart with the epochs left in the run; this replaces the fixed choice about L2 at $T=10$. Verdict R2, Section 6’s Scenario B and Verdict R3, Section 10’s condition (5), Section 13 and the verdict table follow. The new result is numbered after Section 5’s other results, so no earlier number changes.
4. *Space budget.* Theorem 4.2 is stated with a headroom in bytes, so the symbol β now means only the SLO slack; the headroom comes from the box’s space bound, and the 2% figures are kept as an example.
5. *Measurement modes.* Section 9.2 states which results depend on the mode: none of the proofs, but the reading of RA and of the latency propositions, the conditions set by measured timing (notably μ_0 , now calibrated in paper mode), and every measured input. G5, G6, Proposition 9.2, Section 9.3, Section 13’s safety item and the threats list follow.
6. *Learner recommendation.* Section 13’s item 3 names the one-step form with a model-based prior, which Revision 3’s item 3 already allowed; the abstract’s item (iv) notes it.
7. *Phase detection.* The plain-terms text after Proposition 8.1 reports a simulation of the detector on 1,000-operation windows.
8. *Implementation of the multipliers* (Remarks 4.13 and 5.11): the project scales scores rather than rewriting targets, so RocksDB’s stall estimate stays native; the threats list is updated.
9. *Read metric the controller optimises* (M2, Section 3.4): the data-block part RA_d ; table opens are reported separately, because RocksDB’s counters do not attribute them to point lookups or levels.
10. *L1 exception* (Section 6, Scenario B). Revision 3 said gains at L1–L2 must come from dropped versions or timing; that holds only for levels with equal neighbour ratios. L1’s upstream ratio is set by the L0 batch, so resizing it is a first-order static choice (Theorem 4.4), still to be judged against B3.
11. *Labels:* Assumption 5.3’s items are renamed (N1)–(N4), so that O1–O3 name only the project’s decisions.
12. *Register:* item 36 updated; items 37–38 added.

B.2 Changes in Revision 3 (from Revision 2)

1. *Ground rule G7 and the safety envelope.* RocksDB now picks every compaction; the controller acts only through k and per-level multipliers. Section 5.3 replaces Revision 2’s ageing arbiter with an envelope that works through those inputs (Definition 5.8, Theorem 5.9). The arbiter chose among levels itself, which G7 forbids; it also made early compaction impossible, because a level became a candidate only when its native score reached 1; and its no-starvation proof assumed that a level’s candidacy restarts only when that level is picked. The new guarantees are a size bound, freedom from L0 slowdowns and bounded waits with ageing, each under a stated score-gap condition; the unconditional wait bound is given up (Remark 5.10). Definition 4.1, Remark 4.13, the options table, the mitigations of Section 6 and Section 13 follow.
2. *All VERIFY items closed* against RocksDB’s source (main, September 2026): the job split (E2), the `perf_level` values plus the need for `EnablePerLevelPerfContext()` (E7), the pending-bytes formula, the mutex (Proposition 5.13, now VERIFIED and stated for score computation), and the picker’s order.
3. *L0 byte scoring* (new E10): in static mode L0’s score is $\max(n_0/k, B_0/C_1)$, so the effective trigger is $\min(k, \lceil C_1/f_0 \rceil)$. Consequences: only 9 of the 12 (k, base) grid pairs are distinct; Theorem 7.2(iii)’s L0→L1 cost at $k = 16$, base 16 MiB, is 2 (Revision 2 said 1.5); the four-flush margin at $k = 16$ holds only at base 32 MiB.
4. *Stall limits* (new E11, Remark 5.11): 64/128 GiB are `db_bench`’s defaults and do not bind at 2.8 GiB; RocksDB’s estimate uses the patched targets, so S2 also checks native targets.
5. *Theorem 4.4:* states what Δ measures; outside cycles the difference between policy costs is $\Delta - \delta b S'_i(s(H) - s(0))$. The corollaries, which use cycles, are unaffected.
6. *Corollary 4.10:* the AM–GM argument was not what proves it and fails under conservation; replaced by a direct argument, with the note that a grid over T and base approaches the best fixed geometry only from above.
7. *Theorem 6.2:* the counterexample is rebuilt. In Revision 2’s state x' ($n_0 = 19$), L0 already had the top score, so an L3 compaction could not take the slot.
8. *Theorem 7.2(i):* early compaction changes point reads by Corollary 4.11’s full term (false-positive probe plus cache), not only the cache term as Revision 2 said.
9. *Theorem 7.3:* averages are per operation, which the proof needs; “a precise form of the RUM conjecture” softened to “related to”.
10. *Propagation-law numbers:* Table 1 separates the fixed point, which the law would deploy, from the exact conserved optimum, which Revision 2 reported as the law’s. Gain over the best grid point: 0.8%, 1.9%, 3.7% at $\kappa = 0, 0.2, 0.5$ (Revision 2: 1.2–3.7%). Extra gain from knowing κ for fixed points: -0.05% , -0.07% , $+0.5\%$, $+3.3\%$ (Revision 2 quoted the exact optima’s 0.04%, 0.17%, 1.4%, 5.1%). Figure 3 shows both; the abstract is updated. All numbers in Tables 1 and 2 were recomputed.
11. *Attribution:* Section 5.5 credits RusKey’s policy propagation and Lim et al.’s level-size optimisation and states what the law adds; Section 14 adds a novelty item covering ArceKV and EcoTune.
12. *Proposition 9.3* retagged CONDITIONAL with an explicit construction; “identical RA” was not achievable.

13. *Tombstones*: removed from R4’s levers and from the state for G4’s workload, which has no deletes.
14. *Unsupported cross-reference* (“overwrites absorbed at L0–L2”, attributed to Section 11, which does not report it) flagged VERIFY.
15. *Smaller corrections*: Theorem 3.2’s averaged form also assumes α_r independent of the key’s location; Theorem 4.2’s budget is summed over simultaneously expanded levels and the 2% labelled a working assumption; quoted U values state that they omit memtables and L0; (P3) carries the normalising constant c ; the NEXP statement is worded for the general class; the dangling “reflection condition (7)” is fixed; Section 13’s mission-length wording matches its numbers; the two write constraints are stated as open decision O1; the $T=2$ joint action (972 values) is noted; the weight-push interval is defined in operations; Section 5.6 adds the on/off comparison (step 6); Appendix A is renumbered in order.
16. *Citations*: [33] now points to RocksDB’s headers (was the rust-rocksdb documentation); [34] now points to the source function (RocksDB issue #9351 concerned a `fillseq` slowdown and mentioned the flush split only in a comment); [10] gains its pages; E8 cites RocksDB 8.2.0 for the 32 MB default. The claims attributed to [8, 9, 10, 12, 14, 24, 35] were checked against those sources, and the bibliographic data of [13, 18, 20, 22, 23] against publisher or index records; the remaining standard references were checked from bibliographic knowledge.
17. *Section 11* is labelled pre-restart context (open decision O3).

B.3 Changes in Revision 2 (from Revision 1)

1. Dynamic level bytes: Revision 1 assumed `true`; now `false` (G2). Rewritten: targets and scores, the space theorem, the space bound U , the conservation identity (now for unequal ratios), the level-count corollary, and the new B2 baseline.
2. Point reads: Revision 1 modelled only lookups for missing keys, with no cache and all tables open; replaced by Theorem 3.2. Its claim that expanding a level ≥ 1 leaves point reads unchanged was wrong (Corollary 4.11).
3. Learning signal: Revision 1’s heuristic (evidence $\propto T^{-i}$) is replaced by Theorems 5.2 and 5.4; the number of decisions does not fall with depth.
4. Staleness: Revision 1 inferred a change rate from $R \approx 50$ ms, which was the decision interval, not a round trip; withdrawn. Proposition 5.13 and the options table added.
5. Early compaction: “dominated at levels ≥ 1 ” holds only where neighbour ratios are equal; level $L-1$ under static targets is the exception.
6. L0: Revision 1 used RocksDB defaults (256 MB base, 64 MB files); now the project’s configuration and ground rule G3.
7. Break-even figure: Revision 1’s model-based curve omitted the $\delta_K K$ term (ℓ^* shown as 5.5 instead of 5.75); fixed.
8. Secondary figures from Endure, CAMAL, ArceKV and Lethe that were not re-verified were removed; the SILK claim was re-verified against its abstract.
9. Removed as unnecessary or unsupported: “stalls apply to the whole DB (single column family)” and “L0 does not compact below the file trigger” (which conflicts with L0 byte scoring).

10. Added: configuration analysis (Section 2), latency and modes (Section 9), WAL margin algebra, DQN analysis (Section 10), cold-start exploration, and the assumption register.
11. Added: a propagation law for deep-level targets (Section 5.5) with its numerical check and its validation status and plan (Section 5.6).
12. Verdicts restated for the resolved designs; R4(b) corrected from *before* to *at the start of* a read-heavy phase, with its mechanism.
13. Full review: the propagation model now conserves data. Earlier drafts held the last level fixed, which overstated the modelled gain (26% became 16% over native targets) and wrongly credited `dynamic_level_bytes=true` with the write gain above a partly filled last level; the arbiter theorem, the counterexample of Theorem 6.2, the elision bound, the measurement of κ_j and δ_j , the attribution of the $(T + 1)/2$ figure, the NEXP-completeness wording and bibliographic details were corrected.
14. B2 (`dynamic_level_bytes=true`) became a reference row only; B3 the hull over static-target configurations. The observation-model assumptions of Theorem 5.4 were renamed (O1)–(O4).
15. Added: R3 mitigations for both outcomes of the propagation-model validation, including Theorem 6.4 and its check.

References

- [1] P. O’Neil, E. Cheng, D. Gawlick, E. O’Neil. The log-structured merge-tree (LSM-tree). *Acta Informatica* 33(4):351–385, 1996.
- [2] J. L. Bentley, J. B. Saxe. Decomposable searching problems I: Static-to-dynamic transformation. *J. Algorithms* 1(4):301–358, 1980.
- [3] C. Mathieu, C. Staelin, N. E. Young, A. Yousefi. Bigtable merge compaction. arXiv:1407.3008, 2014.
- [4] C. Mathieu, R. Rajaraman, N. E. Young, A. Yousefi. Competitive data-structure dynamization. *SODA* 2021, pp. 2269–2287.
- [5] N. Dayan, M. Athanassoulis, S. Idreos. Monkey: Optimal navigable key-value store. *SIGMOD* 2017, pp. 79–94.
- [6] N. Dayan, S. Idreos. Dostoevsky: Better space-time trade-offs for LSM-tree based key-value stores via adaptive removal of superfluous merging. *SIGMOD* 2018, pp. 505–520.
- [7] N. Dayan, S. Idreos. The log-structured merge-bush & the Wacky continuum. *SIGMOD* 2019, pp. 449–466.
- [8] D. Mo, S. Luo, S. Idreos. How to grow an LSM-tree? Towards bridging the gap between theory and practice. *Proc. ACM Manag. Data* 3(3), Art. 173 (SIGMOD 2025). arXiv:2504.17178.
- [9] D. Mo, F. Chen, S. Luo, C. Shan. Learning to optimize LSM-trees: Towards a reinforcement learning based key-value store for dynamic workloads. *Proc. ACM Manag. Data* 1(3), Art. 213, Nov. 2023 (SIGMOD 2024). arXiv:2308.07013.
- [10] J. Liu, H. Xie, S. Luo. ArceKV: Towards workload-driven LSM-compactions for key-value store under dynamic workloads. *PVLDB* 19(5):958–972, 2026. arXiv:2508.03565.
- [11] S. Sarkar, D. Staratzis, Z. Zhu, M. Athanassoulis. Constructing and analyzing the LSM compaction design space. *PVLDB* 14(11), 2021.

- [12] H. Lim, D. G. Andersen, M. Kaminsky. Towards accurate and fast evaluation of multi-stage log-structured designs. *USENIX FAST* 2016, pp. 149–166.
- [13] C. Luo, M. J. Carey. On performance stability in LSM-based storage systems. *PVLDB* 13(4):449–462, 2019.
- [14] O. Balmau, F. Dinu, W. Zwaenepoel, K. Gupta, R. Chandhiramoorthi, D. Didona. SILK: Preventing latency spikes in log-structured merge key-value stores. *USENIX ATC* 2019, pp. 753–766.
- [15] O. Balmau, D. Didona, R. Guerraoui, W. Zwaenepoel, H. Yuan, A. Arora, K. Gupta, P. Konka. TRIAD: Creating synergies between memory, disk and log in log structured key-value stores. *USENIX ATC* 2017, pp. 363–375.
- [16] R. Sears, R. Ramakrishnan. bLSM: a general purpose log structured merge tree. *SIGMOD* 2012, pp. 217–228.
- [17] S. Sarkar, T. I. Papon, D. Staratzis, M. Athanassoulis. Lethe: A tunable delete-aware LSM engine. *SIGMOD* 2020, pp. 893–908.
- [18] L. Yang, H. Wu, T. Zhang, X. Cheng, F. Li, L. Zou, Y. Wang, R. Chen, J. Wang, G. Huang. Leaper: A learned prefetcher for cache invalidation in LSM-tree based storage engines. *PVLDB* 13(12):1976–1989, 2020.
- [19] Z. Zhu, J. H. Mun, A. Raman, M. Athanassoulis. Reducing Bloom filter CPU overhead in LSM-trees on modern storage devices. *DaMoN* 2021.
- [20] A. Huynh, H. A. Chaudhari, E. Terzi, M. Athanassoulis. Endure: A robust tuning paradigm for LSM trees under workload uncertainty. *PVLDB* 15(8):1605–1618, 2022.
- [21] S. Chatterjee, M. Jagadeesan, W. Qin, S. Idreos. Cosine: a cloud-cost optimized self-designing key-value storage engine. *PVLDB* 15(1):112–126, 2021.
- [22] J. Liu, F. Wang, D. Mo, S. Luo. Structural designs meet optimality: Exploring optimized LSM-tree structures in a colossal configuration space. *Proc. ACM Manag. Data* 2(3) (SIGMOD 2024).
- [23] W. Yu, S. Luo, Z. Yu, G. Cong. CAMAL: Optimizing LSM-trees via active learning. *Proc. ACM Manag. Data* 2(4), Art. 202, Sept. 2024.
- [24] H. Wang, J. Qiu, F. Yuan, H. Zhang. Rethinking the compaction policies in LSM-trees. *Proc. ACM Manag. Data* 3(3), Art. 207, 2025 (SIGMOD 2025).
- [25] M. Athanassoulis, M. S. Kester, L. M. Maas, R. Stoica, S. Idreos, A. Ailamaki, M. Callaghan. Designing access methods: The RUM conjecture. *EDBT* 2016, pp. 461–466.
- [26] Z. Cao, S. Dong, S. Vemuri, D. H. C. Du. Characterizing, modeling, and benchmarking RocksDB key-value workloads at Facebook. *USENIX FAST* 2020, pp. 209–223.
- [27] S. Dong, A. Kryczka, Y. Jin, M. Stumm. RocksDB: Evolution of development priorities in a key-value store serving large-scale applications. *ACM Trans. Storage* 17(4), Art. 26, 2021.
- [28] RocksDB Wiki. Leveled Compaction. <https://github.com/facebook/rocksdb/wiki/Leveled-Compaction> (accessed Sept. 2026).
- [29] RocksDB Wiki. Leveled Compaction, revision d7662f6. <https://github.com/facebook/rocksdb/wiki/Leveled-Compaction/d7662f64851d72d8576300c495778d82f1d630aa>.
- [30] RocksDB Wiki. Write Stalls. <https://github.com/facebook/rocksdb/wiki/Write-Stalls> (accessed Sept. 2026).

- [31] RocksDB Wiki. RocksDB Tuning Guide. <https://github.com/facebook/rocksdb/wiki/RocksDB-Tuning-Guide> (accessed Sept. 2026).
- [32] RocksDB source, main branch (Sept. 2026): `include/rocksdb/{options,advanced_options,statistics,perf_level,perf_context,table}.h`, `db/version_set.cc`, `db/internal_stats.cc`, `db/compaction/compaction_picker_level.cc`, `db/compaction/compaction_job.cc`, `db/db_impl/db_impl_compaction_flush.cc`, `HISTORY.md` (8.2.0: 32MB default block cache; 8.4.0: `level_compaction_dynamic_level_bytes` defaults to `true`). <https://github.com/facebook/rocksdb>.
- [33] RocksDB source, `include/rocksdb/options.h` (`max_background_jobs`, default 2) and `include/rocksdb/advanced_options.h` (`max_bytes_for_level_multiplier_additional`: `std::vector<int>`, static mode only, changeable through `SetOptions()`). <https://github.com/facebook/rocksdb>.
- [34] RocksDB source, `DBImpl::GetBGJobLimits` in `db/db_impl/db_impl_compaction_flush.cc` and the compaction speed-up conditions in `db/column_family.cc`. <https://github.com/facebook/rocksdb>.
- [35] M. Callaghan. RocksDB internals: bytes pending compaction. Small Datum blog, Jan. 2022. <http://smalldatum.blogspot.com/2022/01/rocksdb-internals-bytes-pending.html>.
- [36] J. D. C. Little. A proof for the queuing formula: $L = \lambda W$. *Operations Research* 9(3):383–387, 1961.
- [37] H. L. Van Trees. *Detection, Estimation, and Modulation Theory, Part I*. Wiley, 1968.
- [38] R. S. Sutton, A. G. Barto. *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [39] C. J. C. H. Watkins, P. Dayan. Q-learning. *Machine Learning* 8:279–292, 1992.
- [40] J. N. Tsitsiklis, B. Van Roy. An analysis of temporal-difference learning with function approximation. *IEEE Trans. Automatic Control* 42(5):674–690, 1997.
- [41] S. P. Singh, R. C. Yee. An upper bound on the loss from approximate optimal-value functions. *Machine Learning* 16(3):227–233, 1994.
- [42] V. Mnih et al. Human-level control through deep reinforcement learning. *Nature* 518:529–533, 2015.
- [43] H. van Hasselt, A. Guez, D. Silver. Deep reinforcement learning with double Q-learning. AAAI 2016, pp. 2094–2100.
- [44] T. P. Lillicrap et al. Continuous control with deep reinforcement learning. arXiv:1509.02971, 2015 (ICLR 2016).
- [45] S. J. Bradtke, M. O. Duff. Reinforcement learning methods for continuous-time Markov decision problems. NIPS 7, pp. 393–400, 1995.
- [46] R. S. Sutton, D. Precup, S. Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence* 112(1–2):181–211, 1999.
- [47] D. P. Bertsekas. *Nonlinear Programming*, 2nd ed. Athena Scientific, 1999.
- [48] S. Boyd, L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.
- [49] A. Tavakoli, F. Pardo, P. Kormushev. Action branching architectures for deep reinforcement learning. AAAI 2018.
- [50] E. Altman. *Constrained Markov Decision Processes*. Chapman & Hall/CRC, 1999.
- [51] J. Achiam, D. Held, A. Tamar, P. Abbeel. Constrained policy optimization. ICML 2017.

- [52] C. Tessler, D. J. Mankowitz, S. Mannor. Reward constrained policy optimization. ICLR 2019.
- [53] F. A. Oliehoek, C. Amato. *A Concise Introduction to Decentralized POMDPs*. Springer, 2016.
- [54] D. S. Bernstein, R. Givan, N. Immerman, S. Zilberstein. The complexity of decentralized control of Markov decision processes. *Math. Oper. Res.* 27(4):819–840, 2002.
- [55] L. Buşoniu, R. Babuška, B. De Schutter. A comprehensive survey of multiagent reinforcement learning. *IEEE Trans. SMC, Part C* 38(2):156–172, 2008.
- [56] C. Claus, C. Boutilier. The dynamics of reinforcement learning in cooperative multiagent systems. AAAI 1998, pp. 746–752.
- [57] E. S. Page. Continuous inspection schemes. *Biometrika* 41(1/2):100–115, 1954.
- [58] G. Lorden. Procedures for reacting to a change in distribution. *Ann. Math. Statist.* 42(6):1897–1908, 1971.
- [59] G. V. Moustakides. Optimal stopping times for detecting changes in distributions. *Ann. Statist.* 14(4):1379–1387, 1986.